

Jinan University

Java Programming Lab Report

Major: Computer Science & Technology

School: International School

Student name : 李荣耀

Student number : 2024103975

Date of Submission (mm-dd-yyyy): 06/08/2026

Instructor: Yuxia Sun

Table of Content

LAB 1	DATE: 3/31/2026.....	4
	Problem 1. (1.5).....	4
	Problem 2. (1.9).....	4
	Problem 3. (1.11) (optional).....	6
	Problem 4. (2.5).....	8
	Problem 5. (2.17).....	9
	Problem 6. (2.19) (optional).....	11
LAB 2	DATE: 4/07/2026.....	13
	Problem 1. (3.5).....	14
	Problem 2. (3.31).....	15
	Problem 3. (3.15) (optional).....	18
	Problem 4. (4.9).....	22
	Problem 5. (4.11).....	23
	Problem 6. (4.17) (optional).....	25
LAB 3	DATE: 4/14/2026.....	27
	Problem 1. (5.9).....	27
	Problem 2. (5.29) (optional).....	30
	Problem 3. (6.5).....	33
	Problem 4. (6.11).....	35
	Problem 5. (6.19) (optional).....	37
LAB 4	DATE: 4/21/2026.....	39
	Problem 1. (7.13).....	39
	Problem 2. (7.23) (Optional).....	41
	Problem 3. (8.5).....	42
	Problem 4. (8.27).....	45
	Problem 5. (8.37) (Optional).....	48
LAB 5	DATE: 4/28/2026.....	53
	Problem 1. (9.1).....	53
	Problem 2. (9.5).....	56
	Problem 3. (9.11).....	57
	Problem 4. (9.9) (Optional).....	61
	Problem 5. (9.13) (Optional).....	66

LAB 6	DATE: 5/12/2026.....	69
Problem 1.	(10.1).....	69
Problem 2.	(10.3).....	73
Problem 3.	(10.7).....	79
Problem 4.	(10.9) (Optional).....	85
Problem 5.	(10.13) (Optional).....	87
LAB 7	DATE: 5/16/2026.....	93
Problem 1.	(11.1).....	93
Problem 2.	(11.5).....	97
Problem 3.	(11.11).....	99
Problem 4.	(11.13) (Optional).....	101
Problem 5.	(11.17) (Optional).....	103
LAB 8	DATE: 5/19/2026.....	105
Problem 1.	(12.3).....	105
Problem 2.	(12.7).....	107
Problem 3.	(12.11).....	109
Problem 4.	(12.27) (Optional).....	112
Problem 5.	(12.33) (Optional).....	116
Problem 6.	(13.1).....	118
Problem 7.	(13.7).....	122
Problem 8.	(13.17) (Optional).....	126

.....

LAB 1 **DATE: 3/31/2026**

Student Name: 李荣耀 **Student ID:** 2024103975

Problem 1. (1.5)

1.5 (*Compute expressions*) Write a program that displays the result of

$$\frac{7.5 \times 6.5 - 4.5 \times 3}{47.5 - 5.5}.$$

*** Source Code / Solution :**

```
public class ComputeExpression {
    public static void main(String[] args) {
        System.out.printf("%.3f", compute());
    }

    // compute the expression
    public static double compute() {
        return (7.5 * 6.5 - 4.5 * 3) / (47.5 - 5.5);
    }
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab1> java .\ComputeExpression.java
0.839
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab1> 
```

*** Debugging/Testing:**

Bug1: The digits after dot are too many.

Fix:

```
System.out.printf("%.3f", compute());
```

Use format output.

Problem 2. (1.9)

1.9 (*Area and perimeter of a rectangle*) Write a program that displays the area and perimeter of a rectangle with a width of **5.3** and height of **8.6** using the following formula:

$$area = width \times height$$

$$perimeter = 2 \times (width + height)$$

*** Source Code / Solution :**

```
public class AreaAndPerimeterOfRactangles {
    // class of ractangle
    public static class Ractangle {
        // members of ractangle
        private double height;
        private double width;

        // constructor to initiaize height and width
        public Ractangle(double height, double width) {
            this.height = height;
            this.width = width;
        }

        // compute area
        public double computeArea() {
            return height * width;
        }

        // compute perimeter
        public double computePerimeter() {
            return 2.0 * (height + width);
        }
    }

    public static void main(String[] args) {
        // calculate the area and perimeter of a certain ractangle
        Ractangle rac = new Ractangle(8.6, 5.3);
        System.out.println("Area: " + rac.computeArea());
        System.out.println("Perimeter: " + rac.computePerimeter());
    }
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab1> java .\AreaAnd erimeterOf actangles.java
Area: 45.58
Perimeter: 27.799999999999997
```

*** Debugging/Testing:**

Bug1: Informal formula when calculate perimeter.

Fix:

```
return 2.0 * (height + width);
```

Use 2.0 as parameter rather than 2 to avoid possible type

conflict.

Problem 3. (1.11) (optional)

***1.11** (Population projection) The U.S. Census Bureau projects population based on the following assumptions:

- One birth every 7 seconds
- One death every 13 seconds
- One new immigrant every 45 seconds

Write a program to display the population for each of the next five years. Assume that the current population is 312,032,486, and one year has 365 days. *Hint:* In Java, if two integers perform division, the result is an integer. The fractional part is truncated. For example, $5 / 4$ is 1 (not 1.25) and $10 / 4$ is 2 (not 2.5). To get an accurate result with the fractional part, one of the values involved in the division must be a number with a decimal point. For example, $5.0 / 4$ is 1.25 and $10 / 4.0$ is 2.5 .

*** Source Code / Solution :**

```
public class PopulationProjection {
    // model of population question
    public static class PopulationModel {
        private long popu;
        private double increRate;
        private double decreRate;
        private double time;

        // conductor to initialize population
        public PopulationModel(long iniPopu) {
            popu = iniPopu;
        }

        // get Popu
        public long getPopu() {
            return popu;
        }

        // compute increasing rate
        public void setIncreRate(double birthRate, double immiRate) {
            increRate = birthRate + immiRate;
        }

        // compute decrease rate
        public void setDecreRate(double deathRate) {
            decreRate = deathRate;
        }

        // set certain time(seconds)
    }
}
```

```
        public void setTime(double time) {
            this.time = time;
        }

        // compute the population after certain time(seconds)
        public void computePopu() {
            popu = (long) (popu + ((increRate - decreRate) * time));
        }
    }

    public static void main(String[] args) {
        // initialize the model
        long iniPop = 312_032_486;
        double birthRate = 1.0 / 7;
        double deathRate = 1.0 / 13;
        double immiRate = 1.0 / 45;
        long gap = 365 * 24 * 60 * 60;
        PopulationModel USPopulation = new PopulationModel(iniPop);
        USPopulation.setIncreRate(birthRate, immiRate);
        USPopulation.setDecreRate(deathRate);

        // compute the poplation of each year in next five years
        for (int years = 1; years <= 5; years++) {
            USPopulation.setTime(gap);
            USPopulation.computePopu();
            System.out.println("The population after "
                               + years
                               + " years is : "
                               + USPopulation.getPopu());
        }
    }
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab1> java .\ opulation rojection.java
The population after 1 years is :314812582
The population after 2 years is :317592678
The population after 3 years is :320372774
The population after 4 years is :323152870
The population after 5 years is :325932966
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab1> █
```

*** Debugging/Testing:**

Bug1: The result is wrong.

Fix:

```
private long popu;
```

```
private double increRate;  
private double decreRate;  
private double time;
```

For population, use type long rather than int to avoid overflow.

Problem 4. (2.5)

***2.5** (Financial application: calculate tips) Write a program that reads the subtotal and the gratuity rate and then computes the gratuity and total. For example, if the user enters **10** for subtotal and **12%** for gratuity rate, the program displays **\$1.2** as gratuity and **\$11.2** as total. Here is a sample run:

```
Enter the subtotal and a gratuity rate: 10 12   
The gratuity is $1.2 and total is $11.2
```



*** Source Code / Solution :**

```
import java.util.Scanner;  
  
public class CalculateTips {  
    public static class TipCalculator {  
        private double subtotal;  
        private double gratuityRate;  
  
        // get subtotal  
        public double getSubtotal() {  
            return subtotal;  
        }  
  
        // constructor to initialize subtotal and gratuityRate  
        public TipCalculator(double subtotal, double gratuityRate) {  
            this.subtotal = subtotal;  
            this.gratuityRate = gratuityRate;  
        }  
  
        // calculate tip  
        public double calculateTip() {  
            double tip = subtotal * gratuityRate / 100;  
            subtotal += tip;  
            return tip;  
        }  
    }  
  
    public static void main(String[] args) {  
        // create a Scanner object to read input from the user
```



```
Scanner input = new Scanner(System.in);

System.out.println("Enter the subtotal and a gratuity rate:");
// read the subtotal and gratuity rate from the user
double subtotal = input.nextDouble();
double gratuityRate = input.nextDouble();

// create a TipCalculator object and calculate the tip and
total
TipCalculator calculator = new TipCalculator(subtotal,
gratuityRate);
System.out.println("The gratuity is $" +
calculator.calculateTip()
                    + " and total is $" +
calculator.getSubtotal());
    input.close();
}
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab1> java .\CalculateTips.java
Enter the subtotal and a gratuity rate:
10 12
The gratuity is $1.2 and total is $11.2
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab1> █
```

*** Debugging/Testing:**

Bug1: The format of output.

Fix:

```
System.out.println("The gratuity is $" +
calculator.calculateTip()
                    + " and total is $" +
calculator.getSubtotal());
```

Ensure the correct space around parameters.

Problem 5. (2.17)

***2.17** (Science: wind-chill temperature) How cold is it outside? The temperature alone is not enough to provide the answer. Other factors including wind speed, relative humidity, and sunshine play important roles in determining coldness outside. In 2001, the National Weather Service (NWS) implemented the new wind-chill temperature to measure the coldness using temperature and wind speed. The formula is

$$t_{wc} = 35.74 + 0.6215t_a - 35.75v^{0.16} + 0.4275t_av^{0.16}$$

where t_a is the outside temperature measured in degrees Fahrenheit, v is the speed measured in miles per hour, and t_{wc} is the wind-chill temperature. The formula cannot be used for wind speeds below 2 mph or temperatures below -58°F or above 41°F .

Write a program that prompts the user to enter a temperature between -58°F and 41°F and a wind speed greater than or equal to 2 then displays the wind-chill temperature. Use `Math.pow(a, b)` to compute $v^{0.16}$. Here is a sample run:



Enter the temperature in Fahrenheit between -58°F and 41°F :

5.3

Enter the wind speed (≥ 2) in miles per hour: 6

The wind chill index is -5.56707

*** Source Code / Solution :**

```
import java.util.Scanner;

public class WindChillTemperature {

    public static class WindChillTemperatureCalculator {
        private double outsideTemperature;
        private double speed;
        private double windChillTemperature;

        // conductor to initialize outside temperature and wind speed
        public WindChillTemperatureCalculator(double
outsideTemperature, double speed) {
            this.outsideTemperature = outsideTemperature;
            this.speed = speed;
        }

        // get wind chill temperature
        public double getWindChillTemperature() {
            return windChillTemperature;
        }

        // calculate wind chill temperature
        public void calculateWindTemperature() {
            windChillTemperature = 35.74 + 0.6215 * outsideTemperature
- 35.75 * Math.pow(speed, 0.16) + 0.4275 * outsideTemperature *
Math.pow(speed,0.16);
        }
    }
}
```

```
    }

}

public static void main(String[] args) {
    // create a scanner object to read user input
    Scanner input = new Scanner(System.in);
    System.out.println("Enter the temperature in Fahrenheit between
-58°F and 41°F:");
    double outsideTemperature = input.nextDouble();
    System.out.println("Enter the wind speed (>= 2) in miles per
hour:");
    double speed = input.nextDouble();

    // calculate wind chill temperature
    WindChillTemperatureCalculator result = new
WindChillTemperatureCalculator(outsideTemperature, speed);
    result.calculateWindTemperature();
    System.out.printf("The wind chill index
is %.5f", result.getWindChillTemperature());

    input.close();
}
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab1> java .\ indChillTemperature.java
Enter the temperature in Fahrenheit between -58 F and 41 F:
5.3
Enter the wind speed (>= 2) in miles per hour:
6
The wind chill index is -5.56707
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab1> █
```

*** Debugging/Testing:**

Bug1: The format of fractional part

Fix:

```
System.out.printf("The wind chill index
is %.5f", result.getWindChillTemperature());
```

Use format output to ensure the digits.

Problem 6. (2.19) (optional)

***2.19** (Geometry: area of a triangle) Write a program that prompts the user to enter three points, (x_1, y_1) , (x_2, y_2) , and (x_3, y_3) , of a triangle then displays its area. The formula for computing the area of a triangle is

$$s = (\text{side1} + \text{side2} + \text{side3})/2;$$

$$\text{area} = \sqrt{s(s - \text{side1})(s - \text{side2})(s - \text{side3})}$$

Here is a sample run:



Enter the coordinates of three points separated by spaces

like x1 y1 x2 y2 x3 y3: 1.5 -3.4 4.6 5 9.5 -3.4



The area of the triangle is 33.6

*** Source Code / Solution :**

```
import java.util.Scanner;

public class AreaOfTriangles {
    public static class TrianglesAreaCalculator {
        private double side1, side2, side3;
        private double area;

        // initialize sides by passing x and y coordinates of each
        vertex
        public TrianglesAreaCalculator(double x1, double y1, double x2,
double y2, double x3, double y3) {
            side1 = Math.sqrt(Math.pow(x2 - x1, 2) + Math.pow(y2 - y1,
2));
            side2 = Math.sqrt(Math.pow(x3 - x2, 2) + Math.pow(y3 - y2,
2));
            side3 = Math.sqrt(Math.pow(x1 - x3, 2) + Math.pow(y1 - y3,
2));
        }

        // get area of the triangle
        public double getArea() {
            return area;
        }

        // calculate the area of the triangle
        public void calculateArea() {
            double s = (side1 + side2 + side3) / 2;
            area = Math.sqrt(s * (s - side1) * (s - side2) * (s -
side3));
        }
    }
}
```

```
    }  
}  
  
public static void main(String[] args) {  
    // create a scanner object to read input  
    Scanner input = new Scanner(System.in);  
    System.out.println("Enter the coordinates of three points  
separated by spaces\r\n" +  
                        "like x1 y1 x2 y2 x3 y3:");  
    double x1 = input.nextDouble();  
    double y1 = input.nextDouble();  
    double x2 = input.nextDouble();  
    double y2 = input.nextDouble();  
    double x3 = input.nextDouble();  
    double y3 = input.nextDouble();  
  
    // calculate the area of the triangle  
    TrianglesAreaCalculator result = new  
TrianglesAreaCalculator(x1, y1, x2, y2, x3, y3);  
    result.calculateArea();  
    System.out.printf("The area of the triangle is: %.1f",  
result.getArea());  
  
    input.close();  
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab1> java .\AreaOfTriangles.java  
Enter the coordinates of three points separated by spaces  
like x1 y1 x2 y2 x3 y3:  
1.5 -3.4 4.6 5 9.5 -3.4  
The area of the triangle is: 33.6
```

*** Debugging/Testing:**

Bug1: The incorrect format of output.

Fix:

```
System.out.printf("The area of the triangle is: %.1f",  
result.getArea());
```

Use format output to ensure only one digit after dot.

Student Name: 李荣耀 Student ID: 2024103975

Problem 1. (3.5)

***3.5** (*Find future dates*) Write a program that prompts the user to enter an integer for today's day of the week (Sunday is 0, Monday is 1, . . . , and Saturday is 6). Also prompt the user to enter the number of days after today for a future day and display the future day of the week. Here is a sample run:

Enter today's day: 1
Enter the number of days elapsed since today: 3
Today is Monday and the future day is Thursday



Enter today's day: 0
Enter the number of days elapsed since today: 31
Today is Sunday and the future day is Wednesday



*** Source Code / Solution :**

```
package Lab2;

import java.util.Scanner;

public class FindFutureDate {
    /**
     * print the day according to today and the days after today
     * @param today the day of today
     * @param days the days
     */
    public static void printFutureDay(int today, int days) {
        // error check
        if (today < 0) {
            System.out.println("Error input of today");
            return;
        }

        if (days < 0) {
            System.out.println("Error input of days");
        }

        // calculate the future day
        String[] dayStrings = { "Sunday", "Monday", "Tuesday",
            "Wednesday", "Thursday", "Friday", "Saturday" };
        int futureDay = (today + days) % 7;
        System.out.println("Today is " + dayStrings[today] + " and the
```

```
future day is " + dayStrings[futureDay]);
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter today's day:");
        int today = sc.nextInt();
        System.out.println("Enter the number of days elapsed since
today:");
        int days = sc.nextInt();

        printFutureDay(today, days);

        sc.close();
    }
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab2> java .\FindFutureDate.java
Enter today's day:
1
Enter the number of days elapsed since today:
3
Today is Monday and the future day is Thursday
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab2> java .\FindFutureDate.java
Enter today's day:
0
Enter the number of days elapsed since today:
31
Today is Sunday and the future day is Wednesday
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab2> █
```

*** Debugging/Testing:**

Bug1: The incorrect index to the days in a week and.

Fix:

```
int futureDay = (today + days) % 7;
```

Modify the corresponding relation.

Problem 2. (3.31)

***3.31** (Financials: currency exchange) Write a program that prompts the user to enter the exchange rate from currency in U.S. dollars to Chinese RMB. Prompt the user to enter **0** to convert from U.S. dollars to Chinese RMB and **1** to convert from Chinese RMB to U.S. dollars. Prompt the user to enter the amount in U.S. dollars or Chinese RMB to convert it to Chinese RMB or U.S. dollars, respectively. Here are the sample runs:



```
Enter the exchange rate from dollars to RMB: 6.81 [Enter]
Enter 0 to convert dollars to RMB and 1 vice versa: 0 [Enter]
Enter the dollar amount: 100 [Enter]
$100.0 is 681.0 yuan
```



```
Enter the exchange rate from dollars to RMB: 6.81 [Enter]
Enter 0 to convert dollars to RMB and 1 vice versa: 1 [Enter]
Enter the RMB amount: 10000
10000.0 yuan is $1468.43
```

Programming Exercises

```
Enter the exchange rate from dollars to RMB: 6.81 [Enter]
Enter 0 to convert dollars to RMB and 1 vice versa: 5 [Enter]
Incorrect input
```



*** Source Code / Solution :**

```
package Lab2;

import java.util.Scanner;

public class CurrencyExchange {
    /**
     * print the result of currency exchange
     * @param rate the rate from dollar to yuan
     * @param type the conversion type
     * @param amount the amount of currency
     */
    public static void printCurrencyExchange(double rate, int type,
double amount) {
        // check the input
        if (type != 0 && type != 1) {
            System.out.println("Incorrect input");
            return;
        }
    }
}
```



```
}

    if (rate <= 0) {
        System.out.println("Incorrect input of rate");
        return;
    }

    if (amount < 0) {
        System.out.println("Incorrect input of amount");
        return;
    }

    // calculate the result according to type
    double target;
    if (type == 0) {
        target = amount * rate;
        System.out.println("$" + amount + " is " +
String.format("%.2f", target) + " yuan");
    } else {
        target = amount / rate;
        System.out.println(amount + " yuan is $" +
String.format("%.2f", target));
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.println("Enter the exchange rate from dollars to
RMB:");
    double rate = sc.nextDouble();
    if (rate <= 0) {
        System.out.println("Incorrect input of rate");
        sc.close();
        return;
    }

    System.out.println("Enter 0 to convert dollars to RMB and 1
vice versa:");
    int type = sc.nextInt();
    if (type != 0 && type != 1) {
        System.out.println("Incorrect input");
        sc.close();
        return;
    }

    System.out.println("Enter the RMB amount:");
```

```
double amount = sc.nextDouble();
if (amount < 0) {
    System.out.println("Incorrect input of amount");
    sc.close();
    return;
}

printCurrencyExchange(rate, type, amount);

sc.close();
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab2> java .\CurrencyExchange.java
Enter the exchange rate from dollars to RMB:
6.81
Enter 0 to convert dollars to RMB and 1 vice versa:
0
Enter the RMB amount:
100
$100.0 is 681.00 yuan
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab2> java .\CurrencyExchange.java
Enter the exchange rate from dollars to RMB:
6.81
Enter 0 to convert dollars to RMB and 1 vice versa:
1
Enter the RMB amount:
10000
10000.0 yuan is $1468.43
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab2> java .\CurrencyExchange.java
Enter the exchange rate from dollars to RMB:
6.81
Enter 0 to convert dollars to RMB and 1 vice versa:
5
Incorrect input
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab2> █
```

*** Debugging/Testing:**

Bug1: The sc is not close if the input is valid.

Fix: Add sc.close() in each incorrect situation.

Problem 3. (3.15) (optional)

****3.15** (Game: lottery) Revise Listing 3.8, Lottery.java, to generate a lottery of a three-digit number. The program prompts the user to enter a three-digit number and determines whether the user wins according to the following rules:

1. If the user input matches the lottery number in the exact order, the award is \$12,000.
2. If all digits in the user input match all digits in the lottery number, the award is \$5,000.
3. If one digit in the user input matches a digit in the lottery number, the award is \$2,000.

*** Source Code / Solution :**

```
package Lab2;

import java.util.Scanner;

public class Lottery {
    /**
     * sort the integers in non-decreasing order
     * @param arr the array storing integers
     */
    public static void sortIntegers(int[] arr) {
        int tmp;
        if (arr[0] < arr[1]) {
            tmp = arr[0];
            arr[0] = arr[1];
            arr[1] = tmp;
        }
        if (arr[1] < arr[2]) {
            tmp = arr[1];
            arr[1] = arr[2];
            arr[2] = tmp;
        }
    }

    /**
     * Generate the lottery and print the matching result
     * @param input the input number
     */
    public static void lotteryMatch(int input) {
        if (input < 0 || input > 999) {
            System.out.println("Incorrect input");
            System.exit(1);
        }
        // generate the result(0~999) randomly
        int result = (int) (Math.random() * 1000);
```

```
// separate three digits
int resHundred = result / 100;
int resTen = (result % 100) / 10;
int resOne = (result % 10);

int inputHundred = input / 100;
int inputTen = (input % 100) / 10;
int inputOne = (input % 10);

System.out.println("The result of the lottery is " +
String.format("%03d", result));

// match in the exact order
if (resHundred == inputHundred && resTen == inputTen && resOne
== inputOne) {
    System.out.println("Match in exact order, the award is
$12000.");
}
// match
else {
    int[] res = { resHundred, resTen, resOne };
    int[] inp = { inputHundred, inputTen, inputOne };

    sortIntegers(res);
    sortIntegers(inp);

    int digitLen = 3;
    // the number of number matches
    int match = 0;

    for (int i = 0; i < digitLen; i++) {
        if (res[i] == inp[i]) {
            match++;
        }
    }

    if (match == 3) {
        System.out.println("Match all digits in the lottery
number, the award is $5,000");
    } else if (match != 0) {
        System.out.println("Match a digit in the lottery
number, the award is $2,000");
    } else {
        System.out.println("No match");
    }
}
```

```
    }  
    }  
}  
  
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
    System.out.println("Enter a three-digit number:");  
    int input = sc.nextInt();  
    if (input < 0 || input > 999) {  
        System.err.println("Incorrect input");  
        sc.close();  
        return;  
    }  
  
    lotteryMatch(input);  
  
    sc.close();  
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab2> java .\Lottery.java  
Enter a three-digit number:  
144  
The result of the lottery is 477  
No match  
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab2> java .\Lottery.java  
Enter a three-digit number:  
984  
The result of the lottery is 457  
Match a digit in the lottery number, the award is $2,000
```

*** Debugging/Testing:**

Bug1: The scale of random number is wrong.

Fix:

```
int result = (int) (Math.random() * 1000);
```

Modify the algorithm to get random. Understand the difference between random() in C and Math.random().

Bug2: The situation of two number match is not handled.

Fix:

```
if (match == 3) {  
    System.out.println("Match all digits in the lottery  
number, the award is $5,000");  
} else if (match != 0) {  
    System.out.println("Match a digit in the lottery  
number, the award is $2,000");  
}
```

```
    } else {  
        System.out.println("No match");  
    }  
}
```

Let two matches be the second situation. The award is \$2000.

Problem 4. (4.9)

***4.9** (Find the Unicode of a character) Write a program that receives a character and displays its Unicode. Here is a sample run:

```
Enter a character: E   
The Unicode for the character E is 69
```

*** Source Code / Solution :**

```
package Lab2;  
  
import java.util.Scanner;  
  
public class UnicodeOfCharacters {  
    /**  
     * print the unicode of a character  
     * @param ch the character  
     */  
    public static void printUnicode(char ch) {  
        //force type cast  
        int Unicode = (int) ch;  
        System.out.println("The Unicode for the character " + ch + " is  
" + Unicode);  
    }  
  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        System.out.println("Enter a character:");  
        char ch = sc.next().charAt(0);  
        printUnicode(ch);  
  
        sc.close();  
    }  
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab2> java .\UnicodeOfCharacters.java
Enter a character:
E
The Unicode for the character E is 69
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab2> |
```

*** Debugging/Testing:**

Bug1: Error method to obtain the character.

Fix:

```
char ch = sc.next().charAt(0);
```

Understand the difference of the performance between C and Java.

Problem 5. (4.11)

***4.11** (Binary to decimal) Write a program that prompts the user to enter binary digits and displays its corresponding decimal value. Here are some sample runs:



```
Enter binary digits (0000 to 1111): 0111 Enter
The decimal value is 7
```



```
Enter binary digits (0000 to 1111): 1001 Enter
The decimal value is 9
```



```
Enter binary digits (0000 to 1111): 1100 Enter
The decimal value is 12
```

*** Source Code / Solution :**

```
package Lab2;

import java.util.Scanner;

public class BinaryToDecimal {
    /**
     * transform the binary to decimal(unsigned)
     * @param bitString binary string
     * @return decimal of binary string
     */
    public static int B2U(String bitString) {
        int len = bitString.length();
        // check illegal length
        if (len > 4) {
            System.out.println("Error length of bit string");
            System.exit(1);
        }
    }
}
```

```
int decimal = 0;
int mul = 1;

// multiple corresponding digit and add to result
for (int i = len - 1; i >= 0; i--) {
    char ch = bitString.charAt(i);
    if (ch != '0' && ch != '1') {
        System.out.println("Incorrect input of binary string");
        System.exit(1);
    }
    decimal += (ch - '0') * mul;
    mul *= 2;
}

return decimal;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.println("Enter binary digits (0000 to 1111):");
    String bitString = sc.next();

    System.out.println("The decimal value is " + B2U(bitString));

    sc.close();
}
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab2> java .\BinaryToDecimal.java
Enter binary digits (0000 to 1111):
0111
The decimal value is 7
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab2> java .\BinaryToDecimal.java
Enter binary digits (0000 to 1111):
1001
The decimal value is 9
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab2> java .\BinaryToDecimal.java
Enter binary digits (0000 to 1111):
1100
The decimal value is 12
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab2> █
```

*** Debugging/Testing:**

Bug1: The result is incorrect.

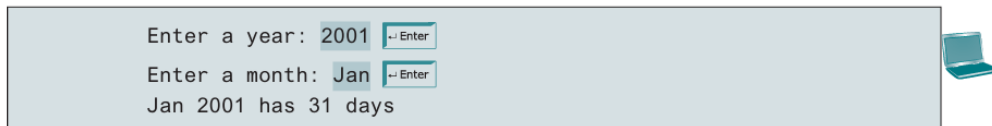
Fix:

```
decimal += (ch - '0') * mul;
```

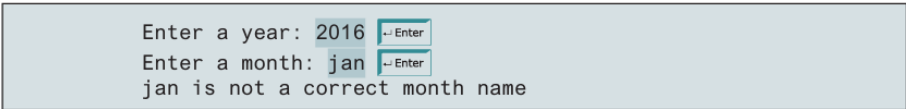

The transformation between the character to its corresponding integer should be `ch - '0'` rather than `type cast(Unicode)`.

Problem 6. (4.17) (optional)

***4.17** (*Days of a month*) Write a program that prompts the user to enter the year and the first three letters of a month name (with the first letter in uppercase) and displays the number of days in the month. If the input for month is incorrect, display a message as presented in the following sample runs:



Chapter 4 Mathematical Functions, Characters, and Strings



*** Source Code / Solution :**

```
package Lab2;

import java.util.Scanner;

public class DaysOfAMonth {

    /**
     * find the days of month in corresponding year
     * @param year
     * @param month
     * @return the days of month in corresponding year
     */
    public static int findDaysOfMonth(int year, String month) {
        int ret = 0;
        // map month to corresponding days
        int[] days = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30,
31 };
        // check whether is leap year
        if (((year % 100 != 0) && (year % 4 == 0)) || (year % 400 ==
0)) {
            days[1] = 29;
        }
    }
}
```

```
}

// map string to corresponding index
String[] months = { "Jan", "Feb", "Mar", "Apr", "May", "Jun",
"Jul", "Aug", "Sep", "Oct", "Nov", "Dec" };
int monthsLen = months.length;
boolean isMonth = false;

for (int i = 0; i < monthsLen; i++) {
    if (month.equals(months[i])) {
        ret = days[i];
        isMonth = true;
    }
}

if (isMonth == false) {
    System.out.println(month + " is not a correct month name");
    System.exit(1);
}

return ret;
}
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.println("Enter a year:");
    int year = sc.nextInt();
    System.out.println("Enter a month:");
    String month = sc.next();

    int days = findDaysOfMonth(year, month);
    System.out.println(month + " " + year + " has " + days + "
days");

    sc.close();
}
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab2> java .\DaysOfAMonth.java
Enter a year:
2001
Enter a month:
Jan
Jan 2001 has 31 days
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab2> java .\DaysOfAMonth.java
Enter a year:
2000
Enter a month:
Feb
Feb 2000 has 29 days
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab2> java .\DaysOfAMonth.java
Enter a year:
2016
Enter a month:
jan
jan is not a correct month name
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab2> █
```

*** Debugging/Testing:**

Bug1: The comparison between strings.

Fix:

```
if (month.equals(months[i])) {
```

The comparison between strings should use compareTo() or equals() rather than directly use '=='

LAB 3 DATE: 4/14/2026

Student Name: 李荣耀 **Student ID:** 2024103975

Problem 1. (5.9)

***5.9** (Find the two lowest scores) Write a program that prompts the user to enter the number of students and each student's name and score, and finally displays the names of the students with the lowest and second-lowest scores.

*** Source Code / Solution :**

```
import java.util.Scanner;

public class FindTwoLowestScores {
    // class of a single student
    public static class Student {
        private String name;
        private double score;

        public Student(String name, double score) {
            this.name = name;
            this.score = score;
        }

        public String getName() {
```

```
        return name;
    }

    public double getScore() {
        return score;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setScore(double score) {
        this.score = score;
    }
}

/**
 * find two lowest scores and print them and corresponding name
 * @param s the array of students
 */
public static void printTwoLowestScores(Student[] s, int num) {
    double[] scores = new double[num];
    int idx1 = 0, idx2 = 0;
    for (int i = 0; i < num; i++) {
        scores[i] = s[i].getScore();
    }

    // find the lowest score
    for (int i = 0; i < num; i++) {
        if (scores[i] < scores[idx1]) {
            idx1 = i;
        }
    }

    // find the second lowest score
    for (int i = 0; i < num; i++) {
        if (scores[i] < scores[idx2] && i != idx1) {
            idx2 = i;
        }
    }

    // print the two lowest scores and corresponding name
    System.out.println("The lowest student: " + s[idx1].getName() +
        "; Score: " + scores[idx1]);
    System.out.println("The second lowest student: " +
```

```
s[idx2].getName() + "; Score: " + scores[idx2]);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Please enter the number of students.");
        int num = sc.nextInt();

        System.out.println("Please enter the name and score of each
student. Like \"Leon 11037\"");
        Student[] s = new Student[num];
        for (int i = 0; i < num; i++) {
            String name = sc.next();
            double score = sc.nextDouble();
            s[i] = new Student(name, score);
        }

        printTwoLowestScores(s, num);
        sc.close();
    }
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab3> java .\FindTwoLowestScores.java
Please enter the number of students.
3
Please enter the name and score of each student. Like "Leon 11037"
Leon 89
Kim 67
Fled 78
The lowest student: Kim; Score: 67.0
The second lowest student: Fled; Score: 78.0
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab3> █
```

*** Debugging/Testing:**

Bug1: The lowest student and the second lowest student is the same one.

Fix:

```
        for (int i = 0; i < num; i++) {
            if (scores[i] < scores[idx2] && i != idx1) {
                idx2 = i;
            }
        }
    }
```

Modify the condition. It should be `i != idx1` rather than `idx2 != idx1`.

Problem 2. (5.29) (optional)

****5.29** (*Display calendars*) Write a program that prompts the user to enter the year and first day of the year and displays the calendar table for the year on the console. For example, if the user entered the year **2013**, and **2** for Tuesday, January 1, 2013, your program should display the calendar for each month in the year, as follows:

January 2013						
Sun	Mon	Tue	Wed	Thu	Fri	Sat
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	31		
...						
December 2013						
Sun	Mon	Tue	Wed	Thu	Fri	Sat
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	31				

*** Source Code / Solution :**

```
import java.util.Scanner;

public class DisplayCalendars {
    /**
     * display the calendar by the year and the first day
     * @param year the year
     * @param firstDay the first day
     */
    public static void displayCalendars(int year, int firstDay) {
        final String[] months = { "January", "February", "March",
            "April", "May", "June", "July", "August", "September",
            "October", "November", "December" };
        int[] daysInMonth = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31,
            30, 31 };

        if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) {
            daysInMonth[1] = 29;
        }
    }
}
```

```
        int currentFirstDay = firstDay;

        for (int m = 0; m < 12; m++) {
            // print the head of the table
            System.out.println("\n" + String.format("%-10s", months[m]
+ " " + year));
            System.out.println("-".repeat(28));
            System.out.printf("%-4s%-4s%-4s%-4s%-4s%-4s%-4s\n", "Sun",
"Mon", "Tue", "Wed", "Thu", "Fri", "Sat");

            // print the former space
            for (int i = 0; i < currentFirstDay; i++) {
                System.out.printf(" ".repeat(4));
            }

            // print dates
            int dayOfWeek = currentFirstDay;
            for (int day = 1; day <= daysInMonth[m]; day++) {
                System.out.printf("%-4d", day);

                if (dayOfWeek == 6) {
                    System.out.println();
                }
                dayOfWeek = (dayOfWeek + 1) % 7;
            }

            System.out.println();

            currentFirstDay = (currentFirstDay + daysInMonth[m]) % 7;
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Please enter the year:");
        int year = sc.nextInt();
        System.out.println("Please enter the first day of the year (0-6
for Sunday-Saturday):");
        int firstDay = sc.nextInt();
        displayCalendars(year, firstDay);

        sc.close();
    }
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab3> java .\DisplayCalendars.java
Please enter the year:
2026
Please enter the first day of the year (0-6 for Sunday-Saturday):
4

January 2026
-----
Sun Mon Tue Wed Thu Fri Sat
                1  2  3
 4  5  6  7  8  9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30 31

February 2026
-----
Sun Mon Tue Wed Thu Fri Sat
 1  2  3  4  5  6  7
 8  9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28

March 2026
-----
Sun Mon Tue Wed Thu Fri Sat
 1  2  3  4  5  6  7
 8  9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
29 30 31

April 2026
-----
Sun Mon Tue Wed Thu Fri Sat
                1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30

May 2026
-----
Sun Mon Tue Wed Thu Fri Sat
                1  2
 3  4  5  6  7  8  9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30
31

June 2026
-----
Sun Mon Tue Wed Thu Fri Sat
 1  2  3  4  5  6
 7  8  9 10 11 12 13
14 15 16 17 18 19 20
21 22 23 24 25 26 27
28 29 30

July 2026
-----
Sun Mon Tue Wed Thu Fri Sat
                1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30 31

August 2026
-----
Sun Mon Tue Wed Thu Fri Sat
                1
 2  3  4  5  6  7  8
 9 10 11 12 13 14 15
16 17 18 19 20 21 22
```



```
23 24 25 26 27 28 29
30 31

September 2026
-----
Sun Mon Tue Wed Thu Fri Sat
          1  2  3  4  5
 6  7  8  9 10 11 12
13 14 15 16 17 18 19
20 21 22 23 24 25 26
27 28 29 30

October 2026
-----
Sun Mon Tue Wed Thu Fri Sat
          1  2  3
 4  5  6  7  8  9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30 31

November 2026
-----
Sun Mon Tue Wed Thu Fri Sat
 1  2  3  4  5  6  7
 8  9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
29 30

December 2026
-----
Sun Mon Tue Wed Thu Fri Sat
          1  2  3  4  5
 6  7  8  9 10 11 12
13 14 15 16 17 18 19
20 21 22 23 24 25 26
27 28 29 30 31
```

*** Debugging/Testing:**

Bug1: The first day of each month is the same.

Fix:

```
currentFirstDay = (currentFirstDay + daysInMonth[m]) % 7;
```

Add a new variable to record the first day of current month.

Problem 3. (6.5)

***6.5** (Sort three numbers) Write a method with the following header to display three numbers in decreasing order:

```
public static void displaySortedNumbers(
    double num1, double num2, double num3)
```

Write a test program that prompts the user to enter three numbers and invokes the method to display them in decreasing order.

*** Source Code / Solution :**

```
import java.util.Scanner;

public class SortThreeNumbers {
    /**
     * display three numbers in decreasing order
     * @param num1 the first number
```

```
    * @param num2 the second number
    * @param num3 the third number
    */
    public static void displaySortedNumbers(double num1, double num2,
double num3) {
        double temp;
        if (num1 < num2) {
            temp = num1;
            num1 = num2;
            num2 = temp;
        }
        if (num1 < num3) {
            temp = num1;
            num1 = num3;
            num3 = temp;
        }
        if (num2 < num3) {
            temp = num2;
            num2 = num3;
            num3 = temp;
        }

        System.out.println(num1);
        System.out.println(num2);
        System.out.println(num3);
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Please enter three numbers: ");
        double num1 = sc.nextDouble();
        double num2 = sc.nextDouble();
        double num3 = sc.nextDouble();

        System.out.println();
        displaySortedNumbers(num1, num2, num3);

        sc.close();
    }
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\Lab3> java .\SortThreeNumbers.java
Please enter three numbers:
22.3
84.2
36.3

84.2
36.3
22.3
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\Lab3>
```

*** Debugging/Testing:**

Bug1: The output and the input are connect.

Fix:

```
System.out.println();
```

Print a new line.

Problem 4. (6.11)

6.11 (Financial application: compute commissions) Write a method that computes the commission, using the scheme in Programming Exercise 5.39. The header of the method is as follows:

```
public static double computeCommission(double salesAmount)
```

Write a test program that displays the following table:

Sales Amount	Commission
10000	900.0
15000	1500.0
20000	2100.0
25000	2700.0
30000	3300.0
35000	3900.0
40000	4500.0
45000	5100.0
50000	5700.0
55000	6300.0
60000	6900.0
65000	7500.0
70000	8100.0
75000	8700.0
80000	9300.0
85000	9900.0
90000	10500.0
95000	11100.0
100000	11700.0

*** Source Code / Solution :**

```
public class ComputeCommissions {
    /**
     * Compute the commission based on the sales amount
     * @param salesAmount
     * @return the commission
     */
    public static double computeCommission(double salesAmount) {
        final double stg1 = 5000.0;
        final double stg2 = 10000.0;
        final double rt1 = 0.06;
        final double rt2 = 0.08;
        final double rt3 = 0.1;
        double commission = 0.0;
        if (salesAmount > 0.0 && salesAmount < stg1) {
            commission = salesAmount * rt1;
        } else if (salesAmount < stg2) {
            commission = stg1 * rt1 + (salesAmount - stg1)
                * rt2;
        } else {
            commission = stg1 * rt1 + (stg2 - stg1) * rt2 +
(salesAmount - stg2) * rt3;
        }

        return commission;
    }
    public static void main(String[] args) {
        // print the head of the table
        System.out.printf("%-15s %-20s\n", "Sales Amount",
"Commission");

        //test the function with some sample sales amounts
        double[] salesAmount = {10000, 15000, 20000, 25000, 30000,
35000, 40000, 45000, 50000, 55000, 60000, 65000,
70000, 75000, 80000, 85000, 90000, 95000, 100000 };

        // print the commission for each sales amount
        for (double ele : salesAmount) {
            System.out.printf("%-15.0f %-20.1f\n", ele,
computeCommission(ele));
        }
    }
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\Lab3> java .\ComputeCommissions.java
Sales Amount      Commission
10000             700.0
15000             1200.0
20000             1700.0
25000             2200.0
30000             2700.0
35000             3200.0
40000             3700.0
45000             4200.0
40000             3700.0
45000             4200.0
50000             4700.0
55000             5200.0
60000             5700.0
65000             6200.0
70000             6700.0
75000             7200.0
80000             7700.0
85000             8200.0
90000             8700.0
95000             9200.0
100000            9700.0
```

*** Debugging/Testing:**

Bug1: The format of output.

Fix:

```
System.out.printf("%-15.0f %-20.1f\n", ele,
computeCommission(ele));
```

Ensure the fixed size and left justifying.

Problem 5. (6.19) (optional)

***6.19** (Triangles) Implement the following two methods:

```
/** Return true if the sum of every two sides is
 * greater than the third side. */
public static boolean isValid(
    double side1, double side2, double side3)

/** Return the area of the triangle. */
public static double area(
    double side1, double side2, double side3)
```

Write a test program that reads three sides for a triangle and uses the **isValid** method to test if the input is valid and uses the **area** method to obtain the area. The program displays the area if the input is valid. Otherwise, it displays that the input is invalid. The formula for computing the area of a triangle is given in Programming Exercise 2.19.

*** Source Code / Solution :**

```
import java.util.Scanner;

public class Triangles {
    /**
```

```
    * Return true if the sum of every two sides greater than the third
    side.
    * @param side1
    * @param side2
    * @param side3
    * @return true or false
    */
    public static boolean isValid(double side1, double side2, double
side3) {
        return (side1 + side2 > side3) && (side2 + side3 > side1) &&
(side1 + side3 > side2);
    }

    /**
    * Return the area of the triangle.
    * @param side1
    * @param side2
    * @param side3
    * @return the area of the triangle
    */
    public static double area(double side1, double side2, double side3)
{
        if (!isValid(side1, side2, side3)) {
            System.out.println("The input is invalid.");
            return 0;
        }

        double s = (side1 + side2 + side3) / 2;
        return Math.sqrt(s * (s - side1) * (s - side2) * (s - side3));
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Please enter the sides of the triangle,
separate by space:");
        double side1 = sc.nextDouble();
        double side2 = sc.nextDouble();
        double side3 = sc.nextDouble();

        if (!isValid(side1, side2, side3)) {
            System.out.println("Invalid input");
            sc.close();
            return;
        }
    }
}
```

```
        System.out.printf("The area of the triangle is: %.2f\n",
area(side1, side2, side3));

        sc.close();
    }
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab3> java .\Triangles.java
Please enter the sides of the triangle, separate by space:
3 4 5
The area of the triangle is: 6.00
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab3> |
```

*** Debugging/Testing:**

Bug1: The format of output.

Fix:

```
        System.out.printf("The area of the triangle is: %.2f\n",
area(side1, side2, side3));
```

Use %.2f to ensure the number of digits.

LAB 4 **DATE: 4/21/2026**

Student Name: 李荣耀 **Student ID:** 2024103975

Problem 1. (7.13)

***7.13** (Random number selector) Write a method that returns a random number from a list of numbers passed in the argument. The method header is specified as follows:

```
public static int getRandom(int... numbers)
```

*** Source Code / Solution :**

```
package Lab4;

public class RandomNumberSelector {
    /**
     * return a random number from a list of numbers
     * @param numbers
     * @return a random number in a list of numbers
     */
    public static int getRandom(int... numbers) {
        // handle argument exception
    }
}
```

```
        if (numbers == null || numbers.length == 0) {
            throw new IllegalArgumentException("Empty array.");
        }
        // generate random index
        int idx = (int) (Math.random() * numbers.length);
        return numbers[idx];
    }

    public static void main(String[] args) {
        int[] numbers = { 34, 13, 78, 29, 91, 114, 514, 1919, 810,
11037 };
        System.out.println("The elements in the array are:");
        for (int ele : numbers) {
            System.out.print(ele + " ");
        }
        System.out.println();

        System.out.println("A random number from the array:");
        System.out.println(getRandom(numbers));
    }
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\La
b4> java .\RandomNumberSelector.java
The elements in the array are:
34 13 78 29 91 114 514 1919 810 11037
A random number from the array:
514
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\La
b4> java .\RandomNumberSelector.java
The elements in the array are:
34 13 78 29 91 114 514 1919 810 11037
A random number from the array:
11037
```

*** Debugging/Testing:**

Bug1: There is no check for array.

Fix:

```
        if (numbers == null || numbers.length == 0) {
            throw new IllegalArgumentException("Empty array.");
        }
```

Throw argument exception if the array is empty. Enhance

the robustness of the program.

Problem 2. (7.23) (Optional)

****7.23** (Game: locker puzzle) A school has 100 lockers and 100 students. All lockers are closed on the first day of school. As the students enter, the first student, denoted as S1, opens every locker. Then the second student, S2, begins with the second locker, denoted as L2, and closes every other locker. Student S3 begins with the third locker and changes every third locker (closes it if it was open and opens it if it was closed). Student S4 begins with locker L4 and changes every fourth locker. Student S5 starts with L5 and changes every fifth locker, and so on, until student S100 changes L100.

After all the students have passed through the building and changed the lockers, which lockers are open? Write a program to find your answer and display all open locker numbers separated by exactly one space.

(Hint: Use an array of 100 Boolean elements, each of which indicates whether a locker is open (**true**) or closed (**false**). Initially, all lockers are closed.)

*** Source Code / Solution :**

```
package Lab4;

public class LockerPuzzle {
    /**
     * solve the locker puzzle
     * @param lockers the array to denote the situation of lockers,
     false is close and true is open
     */
    public static void solveLockerPuzzle(boolean[] lockers){
        int len = lockers.length;
        // i represents i+1-th student
        for (int i = 0; i < len; i++) {
            for (int j = i; j < len; j++) {
                // reverse the situation if satisfying the condition
                if ((j + 1) % (i + 1) == 0) {
                    lockers[j] = !lockers[j];
                }
            }
        }
    }

    public static void main(String[] args) {
        // initialize 100 lockers as false
        final int SIZE = 100;
        boolean[] lockers = new boolean[SIZE];

        solveLockerPuzzle(lockers);
    }
}
```

```
System.out.println("All indices of open lockers are:");

int len = lockers.length;
for (int i = 0; i < len; i++) {
    if (lockers[i]) {
        System.out.print(i + 1 + " ");
    }
}
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\La
b4> java .\LockerPuzzle.java
All indices of open lockers are:
1 4 9 16 25 36 49 64 81 100
```

*** Debugging/Testing:**

Bug1: Null pointer exception.

Fix: Confuse the primitive data type 'boolean' and the class 'Boolean'. Boolean will be initialized as null whereas boolean will be initialized as false in default. If use Boolean, the conditional judgment will throw the null pointer exception.

Problem 3. (8.5)

8.5 (Algebra: add two matrices) Write a method to add two matrices. The header of the method is as follows:

```
public static double[][] addMatrix(double[][] a, double[][] b)
```

In order to be added, the two matrices must have the same dimensions and the same or compatible types of elements. Let **c** be the resulting matrix. Each element c_{ij} is $a_{ij} + b_{ij}$. For example, for two 2×2 matrices **a** and **b**, **c** is

$$\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} + \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix} = \begin{pmatrix} a_{11} + b_{11} & a_{12} + b_{12} \\ a_{21} + b_{21} & a_{22} + b_{22} \end{pmatrix}$$

Write a test program that prompts the user to enter two 2×2 matrices and displays their sum. Here is a sample run:

```
Enter matrix1: 1 2 3 4
Enter matrix2: 0 2 4 1
The matrices are added as follows
1.0 2.0    0.0 2.0    1.0 4.0
3.0 4.0 +  4.0 1.0 =  7.0 5.0
```



*** Source Code / Solution :**

```
package Lab4;

import java.util.Scanner;

public class AddTwoMatrices {
    /**
     * add two matrices
     * @param a the first matrix
     * @param b the second matrix
     * @return the result matrix
     */
    public static double[][] addMatrix(double[][] a, double[][] b) {
        // check whether two matrices have the same DIMensions
        if (a.length != b.length) {
            throw new IllegalArgumentException("Different DIMensions
for matrices.");
        }

        // check whether the matrix is empty
        if (a == null || b == null || a.length == 0 || b.length == 0 ||
a[0] == null || b[0] == null || a[0].length == 0 || b[0].length == 0) {
            throw new IllegalArgumentException("Invalid matrix.");
        }

        // obtain the row and col length
```

```
int row = a.length;
int col = a[0].length;

double[][] res = new double[row][col];

// add two matrices
for (int i = 0; i < row; i++) {
    for (int j = 0; j < col; j++) {
        res[i][j] = a[i][j] + b[i][j];
    }
}

return res;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    final int DIM = 2;
    double[][] matrix1 = new double[DIM][DIM];
    double[][] matrix2 = new double[DIM][DIM];

    // scan two matrices
    System.out.println("Enter matrix1 (2x2):");
    for (int i = 0; i < DIM; i++) {
        for (int j = 0; j < DIM; j++) {
            matrix1[i][j] = sc.nextDouble();
        }
    }

    System.out.println("Enter matrix2 (2x2):");
    for (int i = 0; i < DIM; i++) {
        for (int j = 0; j < DIM; j++) {
            matrix2[i][j] = sc.nextDouble();
        }
    }

    double[][] res = addMatrix(matrix1, matrix2);

    System.out.println("The matrices are added as follows");
    for (int i = 0; i < DIM; i++) {
        // print the same row of each matrix
        for (int j = 0; j < DIM; j++) {
            System.out.printf("%4.1f", matrix1[i][j]);
        }

        if (i == DIM - 1) {
```

```
        System.out.print("  + ");
    } else {
        System.out.print(" ".repeat(4));
    }

    for (int j = 0; j < DIM; j++) {
        System.out.printf("%4.1f", matrix2[i][j]);
    }

    if (i == DIM - 1) {
        System.out.print("  = ");
    } else {
        System.out.print(" ".repeat(4));
    }

    for (int j = 0; j < DIM; j++) {
        System.out.printf("%4.1f", res[i][j]);
    }

    System.out.println();
}

sc.close();
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\Lab4> java .\AddTwo
Matrices.java
Enter matrix1 (2x2):
1 2 3 4
Enter matrix2 (2x2):
0 2 4 1
The matrices are added as follows
1.0 2.0    0.0 2.0    1.0 4.0
3.0 4.0 + 4.0 1.0 = 7.0 5.0
```

*** Debugging/Testing:**

Bug1: The format of output.

Fix: Use three inner loops to respectively print one row of each matrix. Use format output to ensure correct spaces and symbol.

Problem 4. (8.27)

***8.27** (Column sorting) Implement the following method to sort the columns in a two dimensional array. A new array is returned and the original array is intact.

```
public static double[][] sortColumns(double[][] m)
```

apter 8 Multidimensional Arrays

Write a test program that prompts the user to enter a 4×4 matrix of double values and displays a new column-sorted matrix. Here is a sample run:



Enter a 4-by-4 matrix row by row:

0.15 0.875 0.375 0.225

0.55 0.005 0.225 0.015

0.30 0.12 0.4 0.008

0.07 0.021 0.14 0.2

The column-sorted array is

0.07 0.005 0.14 0.008

0.15 0.021 0.225 0.015

0.30 0.12 0.375 0.2

0.55 0.875 0.4 0.225

* Source Code / Solution :

```
package Lab4;

import java.util.Scanner;
import java.util.ArrayList;

public class ColumnSorting {
    /**
     * sort the columns of a matrix
     * @param m
     * @return sorted matrix
     */
    public static double[][] sortColumns(double[][] m) {
        if (m == null || m.length == 0 || m[0] == null || m[0].length
== 0) {
            throw new IllegalArgumentException("Invalid matrix.");
        }

        // obtain row and column length
        int row = m.length;
        int col = m[0].length;
        double[][] res = new double[row][col];
```

```
        for (int j = 0; j < col; j++) {
            // create ArrayList to easily sort
            ArrayList<Double> elements = new ArrayList<Double>();
            for (int i = 0; i < row; i++) {
                elements.add(m[i][j]);
            }

            // sort increasingly
            elements.sort(null);

            // set value to res[i][j]
            for (int i = 0; i < row; i++) {
                res[i][j] = elements.get(i);
            }
        }

        return res;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        final int DIM = 4;
        double[][] matrix = new double[DIM][DIM];

        // scan the matrix
        System.out.println("Enter a 4-by-4 matrix row by row:");
        for (int i = 0; i < DIM; i++) {
            for (int j = 0; j < DIM; j++) {
                matrix[i][j] = sc.nextDouble();
            }
        }

        double[][] res = sortColumns(matrix);

        // output the sorted matrix
        System.out.println("The column-sorted array is");
        for (int i = 0; i < DIM; i++) {
            for (int j = 0; j < DIM; j++) {
                System.out.print(res[i][j] + " ");
            }
            System.out.println();
        }

        sc.close();
    }
}
```

```
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\Lab4> java .\Column
Sorting.java
Enter a 4-by-4 matrix row by row:
0.15 0.875 0.375 0.225
0.55 0.005 0.225 0.015
0.30 0.12 0.4 0.008
0.07 0.021 0.14 0.2
The column-sorted array is
0.07 0.005 0.14 0.008
0.15 0.021 0.225 0.015
0.3 0.12 0.375 0.2
0.55 0.875 0.4 0.225
```

*** Debugging/Testing:**

Bug1: ArrayList cannot be resolved as a type

Fix: Import the package for ArrayList. Use the inner method sort() to sort the array. null as a parameter represents increasing order.

Problem 5. (8.37) (Optional)

****8.37** (*Guess the capitals*) Write a program that repeatedly prompts the user to enter a capital for a state. Upon receiving the user input, the program reports whether the answer is correct. Assume that 50 states and their capitals are stored in a two-dimensional array, as shown in Figure 8.10. The program prompts the user to answer all states' capitals and displays the total correct count. The user's answer is not case-sensitive.

Alabama	Montgomery
Alaska	Juneau
Arizona	Phoenix
...	...
...	...

FIGURE 8.10 A two-dimensional array stores states and their capitals.

Here is a sample run:

```
What is the capital of Alabama? Montgomery ~Enter
The correct answer should be Montgomery
What is the capital of Alaska? Juneau ~Enter
Your answer is correct
What is the capital of Arizona? ...
...
The correct count is 35
```



*** Source Code / Solution :**

```
package Lab4;

import java.util.Scanner;

public class GuessTheCapitals {
    public static void main(String[] args) {
        // two-dimensional array to store states and capitals
        String[][] list =
        {
            {"Alabama", "Montgomery"},
            {"Alaska", "Juneau"},
            {"Arizona", "Phoenix"},
            {"Arkansas", "Little Rock"},
            {"California", "Sacramento"},
            {"Colorado", "Denver"},
            {"Connecticut", "Hartford"},
            {"Delaware", "Dover"},
            {"Florida", "Tallahassee"},
            {"Georgia", "Atlanta"},
            {"Hawaii", "Honolulu"},
            {"Idaho", "Boise"},
            {"Illinois", "Springfield"},
            {"Indiana", "Indianapolis"},
            {"Iowa", "Des Moines"},
            {"Kansas", "Topeka"},
            {"Kentucky", "Frankfort"},
            {"Louisiana", "Baton Rouge"},
            {"Maine", "Augusta"},
            {"Maryland", "Annapolis"},
            {"Massachusetts", "Boston"},
            {"Michigan", "Lansing"},
            {"Minnesota", "St. Paul"},
            {"Mississippi", "Jackson"},
            {"Missouri", "Jefferson City"},
            {"Montana", "Helena"},
            {"Nebraska", "Lincoln"},
            {"Nevada", "Carson City"},
            {"New Hampshire", "Concord"},
            {"New Jersey", "Trenton"},
            {"New Mexico", "Santa Fe"},
            {"New York", "Albany"},
            {"North Carolina", "Raleigh"},
        }
    }
}
```

```
        {"North Dakota", "Bismarck"},
        {"Ohio", "Columbus"},
        {"Oklahoma", "Oklahoma City"},
        {"Oregon", "Salem"},
        {"Pennsylvania", "Harrisburg"},
        {"Rhode Island", "Providence"},
        {"South Carolina", "Columbia"},
        {"South Dakota", "Pierre"},
        {"Tennessee", "Nashville"},
        {"Texas", "Austin"},
        {"Utah", "Salt Lake City"},
        {"Vermont", "Montpelier"},
        {"Virginia", "Richmond"},
        {"Washington", "Olympia"},
        {"West Virginia", "Charleston"},
        {"Wisconsin", "Madison"},
        {"Wyoming", "Cheyenne"}
    };

    Scanner sc = new Scanner(System.in);
    int cnt = 0;

    // tranverse each state and prompt
    for (String[] ele : list) {
        String state = ele[0];
        String capital = ele[1];
        String ans;

        System.out.println("What is the capital of " + state +
" ?");

        ans = sc.nextLine().trim();

        if (ans.equalsIgnoreCase(capital)) {
            System.out.println("Your answer is correct");
            cnt++;
        } else {
            System.out.println("The correct answer should be " +
capital);
        }
    }

    // output the final result
    System.out.println("The correct count is " + cnt);

    sc.close();
```

```
}  
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\Lab4> java .\GuessTheCapitals.java  
What is the capital of Alabama ?  
Montgomery  
The correct answer should be Montgomery  
What is the capital of Alaska ?  
Juneau  
Your answer is correct  
What is the capital of Arizona ?  
Phoenix  
Your answer is correct  
What is the capital of Arkansas ?  
Little Rock  
Your answer is correct  
What is the capital of California ?  
Sacramento  
Your answer is correct  
What is the capital of Colorado ?  
Denver  
The correct answer should be Denver  
What is the capital of Connecticut ?  
Hartford  
The correct answer should be Hartford  
What is the capital of Delaware ?  
Dover  
Your answer is correct  
What is the capital of Florida ?  
Tallahassee  
Your answer is correct  
What is the capital of Georgia ?  
Atlanta  
Your answer is correct  
What is the capital of Hawaii ?  
Honolulu  
Your answer is correct  
What is the capital of Idaho ?  
Boise  
Your answer is correct  
What is the capital of Illinois ?  
Springfield
```

Jinan University– Java Programming Lab Report

```
Your answer is correct
What is the capital of Indiana ?
india
The correct answer should be Indianapolis
What is the capital of Iowa ?
des moines
Your answer is correct
What is the capital of Kansas ?
topeka
Your answer is correct
What is the capital of Kentucky ?
franfkort
The correct answer should be Frankfort
What is the capital of Louisiana ?
baton
The correct answer should be Baton Rouge
What is the capital of Maine ?
augusta
Your answer is correct
What is the capital of Maryland ?
annapolis
Your answer is correct
What is the capital of Massachusetts ?
boston
Your answer is correct
What is the capital of Michigan ?
lansing
Your answer is correct
What is the capital of Minnesota ?
jackson
The correct answer should be St. Paul
What is the capital of Mississippi ?
jakson
The correct answer should be Jackson
What is the capital of Missouri ?
jefferson city
Your answer is correct
What is the capital of Montana ?
helna
The correct answer should be Helena
What is the capital of Nebraska ?
Lincoln
Your answer is correct
What is the capital of Nevada ?
corson city
The correct answer should be Carson City
What is the capital of New Hampshire ?
concord
Your answer is correct
What is the capital of New Jersey ?
trenton
Your answer is correct
What is the capital of New Mexico ?
santa fe
Your answer is correct
What is the capital of New York ?
albany
Your answer is correct
What is the capital of North Carolina ?
releigh
The correct answer should be Raleigh
What is the capital of North Dakota ?
bismarck
Your answer is correct
What is the capital of Ohio ?
columbus
Your answer is correct
What is the capital of Oklahoma ?
oklahoma city
Your answer is correct
What is the capital of Oregon ?
salem
Your answer is correct
What is the capital of Pennsylvania ?
harrisburg
Your answer is correct
What is the capital of Rhode Island ?
amiya
```

```
The correct answer should be Providence
What is the capital of South Carolina ?
you
The correct answer should be Columbia
What is the capital of South Dakota ?
are
The correct answer should be Pierre
What is the capital of Tennessee ?
saying
The correct answer should be Nashville
What is the capital of Texas ?
austin
Your answer is correct
What is the capital of Utah ?
idk
The correct answer should be Salt Lake City
What is the capital of Vermont ?
idk
The correct answer should be Montpelier
What is the capital of Virginia ?
idk 2
The correct answer should be Richmond
What is the capital of Washington ?
Olympia
Your answer is correct
What is the capital of West Virginia ?
idk
The correct answer should be Charleston
What is the capital of Wisconsin ?
where
The correct answer should be Madison
What is the capital of Wyoming ?
idk
The correct answer should be Cheyenne
The correct count is 29
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\Lab4>
```

*** Debugging/Testing:**

Bug1: Jump over the next input when the string of capital includes space.

Fix: Use `nextLine()` rather than `next()` to avoid the input separated by the space. And use `trim()` to eliminate the control characters in the head and tail.

LAB 5 **DATE: 4/28/2026**

Student Name: 李荣耀 **Student ID:** 2024103975

Problem 1. (9.1)

9.1 (The **Rectangle** class) Following the example of the **Circle** class in Section 9.2, design a class named **Rectangle** to represent a rectangle. The class contains:

- Two **double** data fields named **width** and **height** that specify the width and height of the rectangle. The default values are **1** for both **width** and **height**.
- A no-arg constructor that creates a default rectangle.
- A constructor that creates a rectangle with the specified **width** and **height**.
- A method named **getArea()** that returns the area of this rectangle.
- A method named **getPerimeter()** that returns the perimeter.

Draw the UML diagram for the class then implement the class. Write a test program that creates two **Rectangle** objects—one with width **4** and height **40**, and

the other with width **3.5** and height **35.9**. Display the width, height, area, and perimeter of each rectangle in this order.

Programming Exercises

* Source Code / Solution :

```
package Lab5;

public class TheRectangleClass {
    public static class Rectangle {
        private double width = 1;
        private double height = 1;

        // no-arg constructor to create a default rectangle
        public Rectangle() {
        };

        // constructor with specified width and height
        public Rectangle(double width, double height) {
            this.width = width;
            this.height = height;
        }

        public void setWidth(double width) {
            this.width = width;
        }

        public void setHeight(double height) {
            this.height = height;
        }
    }
}
```

```
public double getWidth() {
    return width;
}

public double getHeight() {
    return height;
}

// method to get area
public double getArea() {
    return width * height;
}

// method to get perimeter
public double getPerimeter() {
    return 2 * (width + height);
}

}

public static void main(String[] args) {
    Rectangle rec1 = new Rectangle(4, 40);
    Rectangle rec2 = new Rectangle(3.5, 35.9);

    System.out.println("The width, height, area, and perimeter"
        + "of the first rectangle are "
        + rec1.getWidth() + " "
        + rec1.getHeight() + " "
        + rec1.getArea() + " "
        + rec1.getPerimeter());

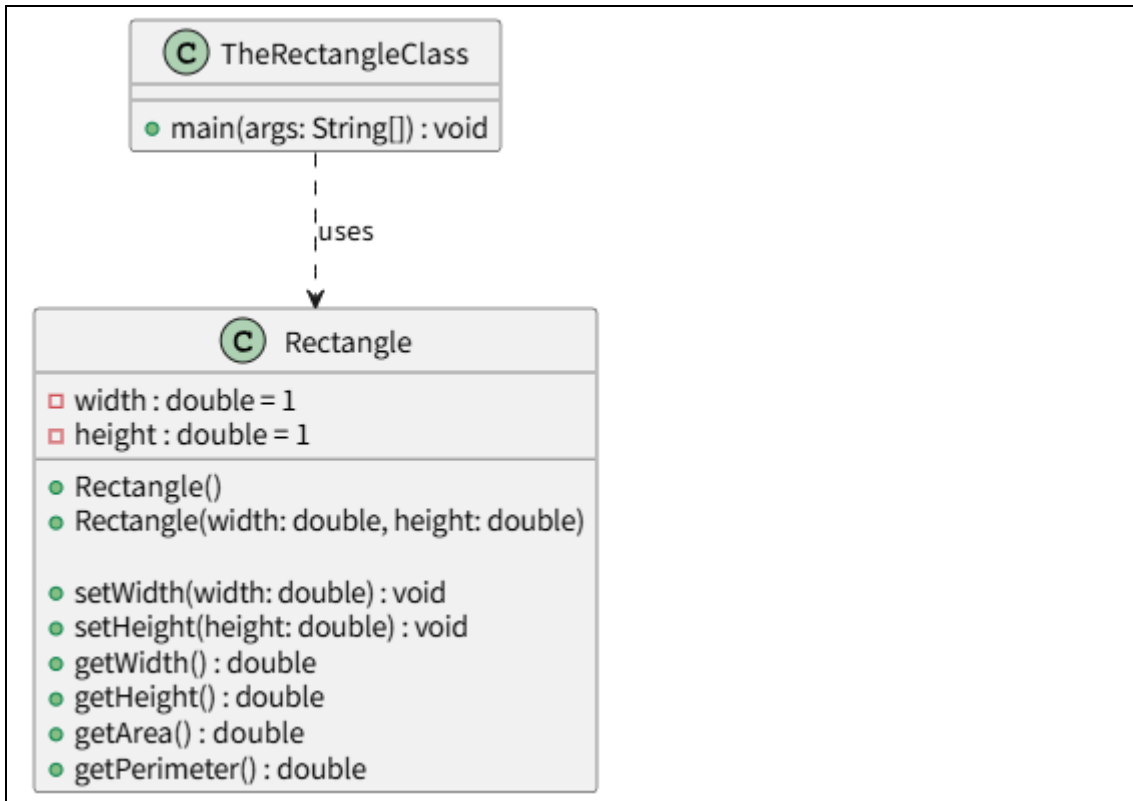
    System.out.println("The width, height, area, and perimeter"
        + "of the first rectangle are"
        + rec2.getWidth() + " "
        + rec2.getHeight() + " "
        + rec2.getArea() + " "
        + rec2.getPerimeter());

}
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\Lab5> java .\TheRectangleClass.java
The width, height, area, and perimeterof the first rectangle are 4.0 40.0 160.0 88.0
The width, height, area, and perimeterof the first rectangle are3.5 35.9 125.64999999999999 78.8
```

The UML graph:



*** Debugging/Testing:**

Bug1: The output format is incorrect.

Fix: Add the space after each parameter.

Problem 2. (9.5)

***9.5** (Use the **GregorianCalendar** class) Java API has the **GregorianCalendar** class in the **java.util** package, which you can use to obtain the year, month, and day of a date. The no-arg constructor constructs an instance for the current date, and the methods **get(GregorianCalendar.YEAR)**, **get(GregorianCalendar.MONTH)**, and **get(GregorianCalendar.DAY_OF_MONTH)** return the year, month, and day. Write a program to perform two tasks:

1. Display the current year, month, and day.
2. The **GregorianCalendar** class has the **setTimeInMillis(long)**, which can be used to set a specified elapsed time since January 1, 1970. Set the value to **1234567898765L** and display the year, month, and day.

*** Source Code / Solution :**

```

package Lab5;

import java.util.GregorianCalendar;

public class UseGregorianCalendar {
    public static void main(String[] args) {
        // create object of current time
    }
}
    
```



```
GregorianCalendar current = new GregorianCalendar();

System.out.println("Current year, month and day:");
System.out.println((current.get(GregorianCalendar.MONTH) + 1) +
"/"
+ current.get(GregorianCalendar.DAY_OF_MONTH) + "/"
+ current.get(GregorianCalendar.YEAR));

// create object of the time after 1234567898765 millis
GregorianCalendar obj = new GregorianCalendar();
obj.setTimeInMillis(1234567898765L);
System.out.println("The time after 1234567898765 millis from
1/1/1970 is:");
System.out.println((obj.get(GregorianCalendar.MONTH) + 1) + "/"
+ obj.get(GregorianCalendar.DAY_OF_MONTH) + "/"
+ obj.get(GregorianCalendar.YEAR));
}
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\Lab5> java .\UseGregorainCalendar.java
Current year, month and day:
4/28/2026
The time after 1234567898765 millis from 1/1/1970 is:
2/14/2009
```

*** Debugging/Testing:**

Bug1: The incorrect order of year, month and day.
Fix: Modify the order of parameters.
Bug2: The month is smaller one month than actual month.
Fix: The GregorainCalendar return the month based on 0.
So the correct output should add 1.

Problem 3. (9.11)

***9.11** (Algebra: 2×2 linear equations) Design a class named **LinearEquation** for a 2×2 system of linear equations:

$$\begin{array}{rcl} ax + by = e & & \\ cx + dy = f & x = \frac{ed - bf}{ad - bc} & y = \frac{af - ec}{ad - bc} \end{array}$$

The class contains:

apter 9 Objects and Classes

- Private data fields **a**, **b**, **c**, **d**, **e**, and **f**.
- A constructor with the arguments for **a**, **b**, **c**, **d**, **e**, and **f**.
- Six getter methods for **a**, **b**, **c**, **d**, **e**, and **f**.
- A method named **isSolvable()** that returns true if $ad - bc$ is not 0.
- Methods **getX()** and **getY()** that return the solution for the equation.

Draw the UML diagram for the class then implement the class. Write a test program that prompts the user to enter **a**, **b**, **c**, **d**, **e**, and **f** and displays the result. If $ad - bc$ is 0, report that “The equation has no solution.” See Programming Exercise 3.3 for sample runs.

* Source Code / Solution :

```
package Lab5;

import java.util.Scanner;

public class LinearEquations {
    public static class LinearEquation {
        private double a, b, c, d, e, f;

        // constructor with arguments
        public LinearEquation(double a, double b, double c,
double d, double e, double f) {
            this.a = a;
            this.b = b;
            this.c = c;
            this.d = d;
            this.e = e;
            this.f = f;
        }

        // getter methods
        public double getA() {
```

```
        return a;
    }

    public double getB() {
        return b;
    }

    public double getC() {
        return c;
    }

    public double getD() {
        return d;
    }

    public double getE() {
        return e;
    }

    public double getF() {
        return f;
    }

    // return true if ad-bc is not 0
    public boolean isSolvable() {
        return (a * d - b * c != 0);
    }

    // get x if it is solvable
    public double getX() {
        if (isSolvable()) {
            return (e * d - b * f) / (a * d - b * c);
        } else {
            throw new ArithmeticException("The equation has
no solution.");
        }
    }

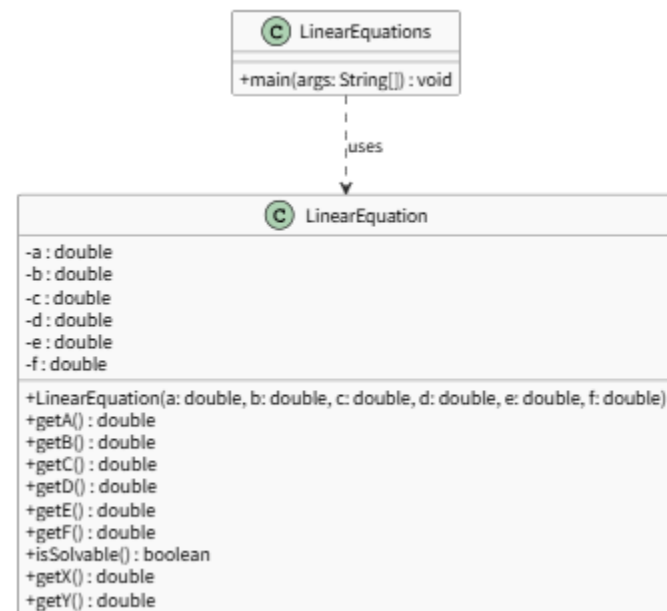
    // get y if it is solvable
    public double getY() {
        if (isSolvable()) {
            return (a * f - e * c) / (a * d - b * c);
        } else {
```

```
        throw new ArithmeticException("The equation has  
no solution.");  
    }  
}  
  
public static void main(String[] args) {  
    System.out.println("Please enter the parameters of 2x2  
system of linear equations.");  
    Scanner sc = new Scanner(System.in);  
  
    double a = sc.nextDouble();  
    double b = sc.nextDouble();  
    double c = sc.nextDouble();  
    double d = sc.nextDouble();  
    double e = sc.nextDouble();  
    double f = sc.nextDouble();  
  
    LinearEquation obj = new LinearEquation(a, b, c, d, e,  
f);  
  
    System.out.println("The solution of the linear  
equations:");  
    System.out.println("x: " + obj.getX());  
    System.out.println("y: " + obj.getY());  
  
    sc.close();  
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\Lab5> java .\Linear  
Equations.java  
Please enter the parameters of 2x2 system of linear equations.  
1 1 1 1 1 1  
The solution of the linear equations:  
Exception in thread "main" java.lang.ArithmeticException: The equation has no solution  
.  
    at Lab5.LinearEquations$LinearEquation.getX(LinearEquations.java:54)  
    at Lab5.LinearEquations.main(LinearEquations.java:81)  
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\Lab5> java .\Linear  
Equations.java  
Please enter the parameters of 2x2 system of linear equations.  
3 4 2 1 4 5  
The solution of the linear equations:  
x: 3.2  
y: -1.4
```

The UML graph:



*** Debugging/Testing:**

Bug1: No exception handling in `getX()` and `getY()` if it is not solvable.

Fix: Throw arithmetic exception when it is not solvable.

Problem 4. (9.9) (Optional)

****9.9** (Geometry: *n*-sided regular polygon) In an *n*-sided regular polygon, all sides have the same length and all angles have the same degree (i.e., the polygon is both equilateral and equiangular). Design a class named **RegularPolygon** that contains:

- A private **int** data field named **n** that defines the number of sides in the polygon with default value **3**.

Programming Exercise

- A private **double** data field named **side** that stores the length of the side with default value **1**.
- A private **double** data field named **x** that defines the *x*-coordinate of the polygon's center with default value **0**.
- A private **double** data field named **y** that defines the *y*-coordinate of the polygon's center with default value **0**.
- A no-arg constructor that creates a regular polygon with default values.
- A constructor that creates a regular polygon with the specified number of sides and length of side, centered at **(0, 0)**.
- A constructor that creates a regular polygon with the specified number of sides, length of side, and *x*- and *y*-coordinates.
- The accessor and mutator methods for all data fields.
- The method **getPerimeter()** that returns the perimeter of the polygon.
- The method **getArea()** that returns the area of the polygon. The formula for computing the area of a regular polygon is

$$\text{Area} = \frac{n \times s^2}{4 \times \tan\left(\frac{\pi}{n}\right)}.$$

Draw the UML diagram for the class then implement the class. Write a test program that creates three **RegularPolygon** objects, created using the no-arg constructor, using **RegularPolygon(6, 4)**, and using **RegularPolygon(10, 4, 5.6, 7.8)**. For each object, display its perimeter and area.

* Source Code / Solution :

```
package Lab5;

public class NsideRegularPolygon {
    public static class RegularPolygon {
        private int n = 3;
        private double side = 1;
        private double x = 0;
        private double y = 0;
```

```
// no-arg constructor
public RegularPolygon() {
}

// constructor to create a regular polygon with specified
n, side and default x, y
public RegularPolygon(int n, double side) {
    this.n = n;
    this.side = side;
}

// constructor to create a regular polygon with specified
n, side, x and y
public RegularPolygon(int n, double side, double x,
double y) {
    this.n = n;
    this.side = side;
    this.x = x;
    this.y = y;
}

// accessors and mutators for all data fields
public int getN() {
    return n;
}

public double getSide() {
    return side;
}

public double getX() {
    return x;
}

public double getY() {
    return y;
}

public void setN(int n) {
    this.n = n;
}
```

```
public void setSide(double side) {
    this.side = side;
}

public void setX(double x) {
    this.x = x;
}

public void setY(double y) {
    this.y = y;
}

// return the perimeter of the polygon
public double getPerimeter() {
    return n * side;
}

// return the area of the polygon
public double getArea() {
    return (n * side * side) / (4 * Math.tan(Math.PI /
n));
}

}

public static void main(String[] args) {
    RegularPolygon obj1 = new RegularPolygon();
    RegularPolygon obj2 = new RegularPolygon(6, 4);
    RegularPolygon obj3 = new RegularPolygon(10, 4, 5.6,
7.8);

    System.out.println("The parimeter and area of the first
regular polygon");
    System.out.printf("Perimeter: %.2f; Area: %.3f\n",
obj1.getPerimeter(), obj1.getArea());

    System.out.println("The parimeter and area of the second
regular polygon");
    System.out.printf("Perimeter: %.2f; Area: %.3f\n",
obj2.getPerimeter(), obj2.getArea());

    System.out.println("The parimeter and area of the third
regular polygon");
    System.out.printf("Perimeter: %.2f; Area: %.3f\n",
```

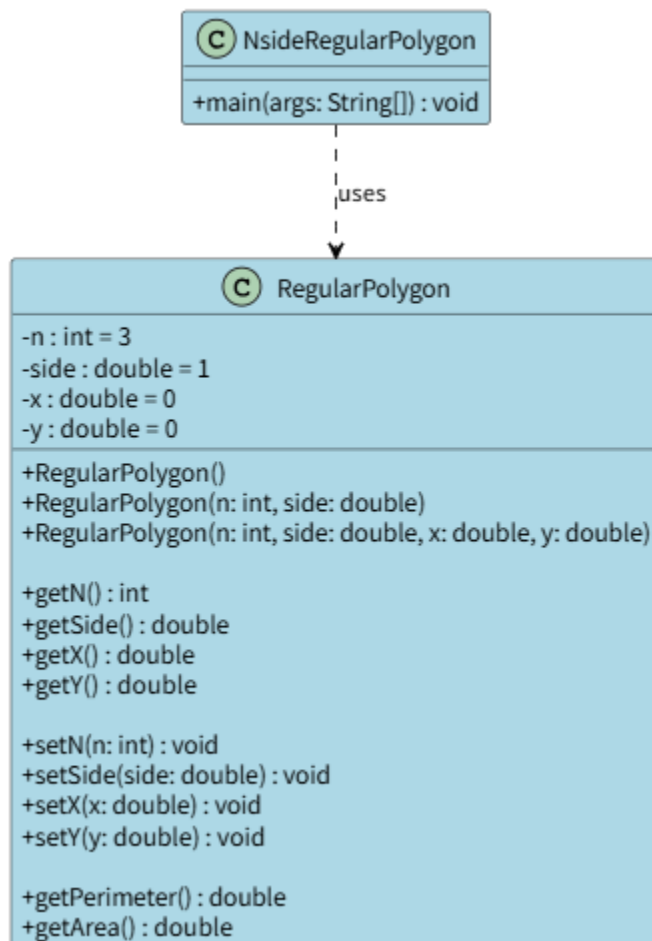


```
obj3.getPerimeter(), obj3.getArea());  
    }  
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\Lab5> java .\NsideR  
egularPolygon.java  
The parimeter and area of the first regular polygon  
Perimeter: 3.00; Area: 0.433  
The parimeter and area of the second regular polygon  
Perimeter: 24.00; Area: 41.569  
The parimeter and area of the third regular polygon  
Perimeter: 40.00; Area: 123.107
```

The UML graph:



*** Debugging/Testing:**

Bug1: Ignore the type of return value of `Math.tan()` and the result of perimeter and area are double, causing too many digits in results.

Fix: Optimize the format of output.

Problem 5. (9.13) (Optional)

****9.13** (The *Location* class) Design a class named *Location* for locating a maximal value and its location in a two-dimensional array. The class contains public data fields *row*, *column*, and *maxValue* that store the maximal value and its indices in a two-dimensional array with *row* and *column* as *int* types and *maxValue* as a *double* type.

Write the following method that returns the location of the largest element in a two-dimensional array:

```
public static Location locateLargest(double[][] a)
```

The return value is an instance of *Location*. Write a test program that prompts the user to enter a two-dimensional array and displays the location of the largest element in the array. Here is a sample run:



```
Enter the number of rows and columns in the array: 3 4 Enter
Enter the array:
23.5 35 2 10 Enter
4.5 3 45 3.5 Enter
35 44 5.5 9.6 Enter
The location of the largest element is 45 at (1, 2)
```

*** Source Code / Solution :**

```
package Lab5;

import java.util.Scanner;

public class TheLocationClass {
    public static class Location {
        public int row;
        public int column;
        public double maxValue;

        // default constructor
        public Location() {
        }

        // constructor to set row and column
        public Location(int row, int column) {
        }
    }
}
```

```
        this.row = row;
        this.column = column;
    }
}

// return the location of the largest element
public static Location locateLargest(double[][] a) {
    if (a.length == 0 || a[0].length == 0 || a == null || a[0] ==
null) {
        throw new IllegalArgumentException("Error two-dimensional
array.");
    }
    Location ret = new Location();

    int row = a.length;
    int col = a[0].length;

    int i = 0, j = 0;
    int maxRow = 0, maxCol = 0;
    double max = a[0][0];

    // find the max value
    for (i = 0; i < row; i++) {
        for (j = 0; j < col; j++) {
            if (a[i][j] > max) {
                max = a[i][j];
                maxRow = i;
                maxCol = j;
            }
        }
    }

    // assign row, column and maxvalue to
    ret.row = maxRow;
    ret.column = maxCol;
    ret.maxValue = max;

    return ret;
}

public static void main(String[] args) {
    System.out.println("Enter the number of rows and columns in the
array");

    Scanner sc = new Scanner(System.in);
}
```

```
int row = sc.nextInt();
int column = sc.nextInt();
double[][] arr = new double[row][column];

System.out.println("Enter the array:");
for (int i = 0; i < row; i++) {
    for (int j = 0; j < column; j++) {
        arr[i][j] = sc.nextDouble();
    }
}

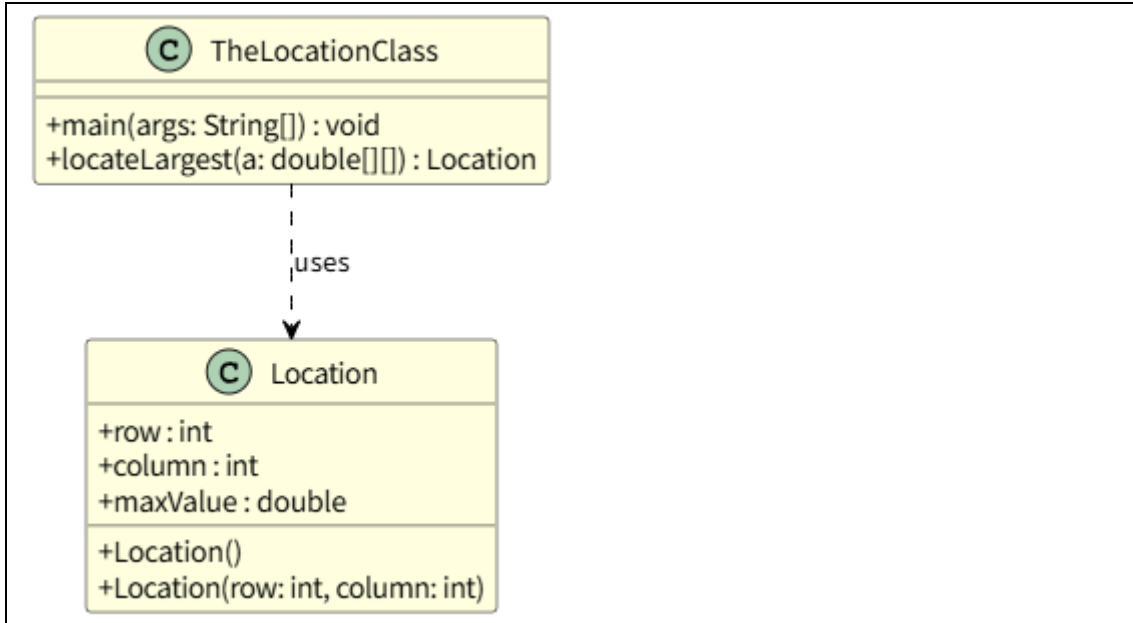
Location maxLocation = new Location();
maxLocation = locateLargest(arr);
System.out.println("The location of the largest element is " +
maxLocation.maxValue + " at (" + maxLocation.row
    + ", " + maxLocation.column + ")");

sc.close();
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab5> java .\TheLocationClass.java
Enter the number of rows and columns in the array
3 4
Enter the array:
23.5 35 2 10
4.5 3 45 3.5
35 44 5.5 9.6
The location of the largest element is 45.0 at (1, 2)
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\lab5> java .\TheLocationClass.java
Enter the number of rows and columns in the array
2 2
Enter the array:
3 4
1 2
The location of the largest element is 4.0 at (0, 1)

The UML graph:
```



*** Debugging/Testing:**

Bug1: The max position is always (0,0)

Fix: We incorrectly use 'break' in the inner loop and the total loop will continue. Thus we use two more variables 'maxRow' and 'maxCol' to record the max position.

Bug2: The max position is still always (0,0)

Fix: We incorrectly set row and col in `locateLargest(double[][] a)`, which causes the loop to find the largest terminates directly. Thus we correctly assign `a.length` and `a[0].length` to row and col.

LAB 6 **DATE: 5/12/2026**

Student Name: 李荣耀 **Student ID:** 2024103975

Problem 1. (10.1)

***10.1** (The **WeatherStation** class) Design a class named **WeatherStation**. The class contains:

- The data fields **temperature** (in °C) and **pressure** (in bar).
- A no-arg constructor that creates a **WeatherStation** where temperature is 0 (°C) and pressure is 1 (bar).
- A constructor that constructs a **WeatherStation** object with particular temperature (in °C) and pressure (in bar).

er 10 Object-Oriented Thinking

- A static method that constructs a **WeatherStation** object with imperial unit values, so temperature in °F and pressure in PSI (It should automatically convert to the valid metric representation before assigning to the data fields.)
- Two getter methods for the data fields **temperature** and **pressure**.
- Two getter methods that convert the data fields to corresponding imperial units (°F and PSI).
- Methods named **setTemperature(double temperature)** and **setPressure(double pressure)** that set the data fields using metric values.
- Methods named **setTemperatureFahrenheit(double temperature)** and **setPressurePSI(double pressure)** that set the data fields using imperial values (They should automatically convert to the underlying metric representation.)

Draw the UML diagram for the class, then implement the class. Write a test programs that creates three **WeatherStation** objects (using **new WeatherStation()**, **new WeatherStation(35, 0.5)**, and **WeatherStation.fromImperial(68, 21)**) and displays their temperature and pressure, in both metric units (°C and bar) and imperial units (°F and PSI).

* Source Code / Solution :

```
package Lab6;

public class WeatherStationTest {
    public static class WeatherStation {

        private double temperature;
        private double pressure;

        // no-arg constructor
        public WeatherStation() {
            temperature = 0;
            pressure = 1;
        }

        // constructor with temperature and pressure
        public WeatherStation(double temperature, double pressure) {
            this.temperature = temperature;
        }
    }
}
```

```
        this.pressure = pressure;
    }

    // static method to construct WeatherStation
    public static WeatherStation ImperialUnitConstructor(double
temperature, double pressure) {
        double temp = temperature * 9 / 5 + 32;
        double pres = pressure / 14.5037738;
        return new WeatherStation(temp, pres);
    }

    // Metric getters
    public double getTemperature() {
        return temperature;
    }

    public double getPressure() {
        return pressure;
    }

    // Imperial getters
    public double getTemperatureFahrenheit() {
        return temperature * 9 / 5 + 32;
    }

    public double getPressurePSI() {
        return pressure * 14.5037738;
    }

    // Metric setters
    public void setTemperature(double temperature) {
        this.temperature = temperature;
    }

    public void setPressure(double pressure) {
        this.pressure = pressure;
    }

    // Imperial setters
    public void setTemperatureFahrenheit(double temperature) {
        this.temperature = (temperature - 32) * 5.0 / 9.0;
    }

    public void setPressurePSI(double pressure) {
        this.pressure = pressure / 14.5037738;
    }
}
```

```
    }  
}  
  
/**  
 * display the data of WeatherStation  
 * @param1  
 * @param2 the object  
 */  
public static void DisplayWeatherStaton(WeatherStation station) {  
    System.out.printf("Metric: %.2f, %.4f\n",  
        station.getTemperature(), station.getPressure());  
    System.out.printf("Imperial: %.2f, %.2f\n",  
        station.getTemperatureFahrenheit(),  
station.getPressurePSI());  
    System.out.println();  
}  
public static void main(String[] args) {  
    WeatherStation station1 = new WeatherStation();  
    WeatherStation station2 = new WeatherStation(35, 0.5);  
    WeatherStation station3 =  
WeatherStation.ImperialUnitConstructor(68, 21);  
  
    System.out.println("The data of these stations are:");  
    System.out.println("Station 1:");  
    DisplayWeatherStaton(station1);  
    System.out.println("Station 2:");  
    DisplayWeatherStaton(station2);  
    System.out.println("Station 3:");  
    DisplayWeatherStaton(station3);  
}  
}
```

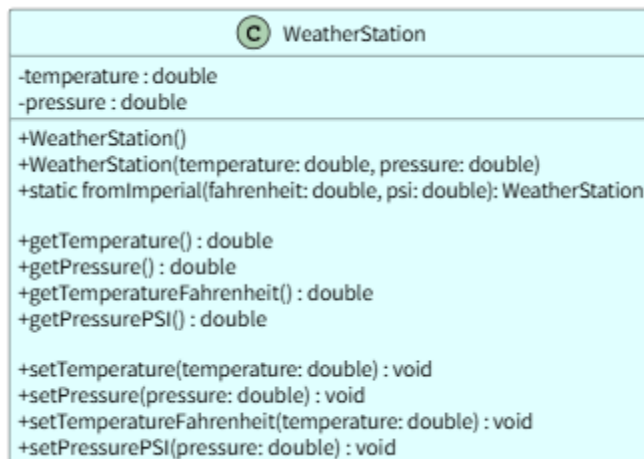
*** Output:**


```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\Lab6> java .\WeatherStationTest.java
The data of these stations are:
Station 1:
Metric: 0.00, 1.0000
Imperial: 32.00, 14.50

Station 2:
Metric: 35.00, 0.5000
Imperial: 95.00, 7.25

Station 3:
Metric: 154.40, 1.4479
Imperial: 309.92, 21.00
```

UML graph:



* Debugging/Testing:

Bug1:

```
WeatherStation station3 = ImperialUnitConstructor(68, 21);
```

Directly invoke the static method to construct an object.

Fix: Use correct syntax to invoke the static constructor. It must be invoked by clearing the class. That is, `className.staticConstructor()`.

Bug2: Repetitive rate is too high when displaying the weather station

Fix: Use a method to display weather station.

Problem 2. (10.3)

10.3 (The **Rational** class) Design a class named **Rational**. The class contains:

- Two **int** data fields that represent the **numerator** and **denominator**.
- A constructor that constructs a **Rational** object for the specified numerator and denominator.
- Getter methods that respectively return the numerator and denominator.
- The method **normalize()** that returns a new **Rational** object that has the same decimal value as the this **Rational** object, but where the numerator is not divisible by the denominator. (Formally, it converts a value $\frac{p1}{q1}$ to the equivalent value $\frac{p2}{q2}$ so $p2$ is not divisible by $q2$. Hint: you need to find the greatest common divisor of the numerator and denominator fields of the current **Rational** to implement this.)
- The methods **isWhole()** and **isDecimal()** that return **true** that check whether the rational is a whole number or has decimals, respectively.
- The methods **equals(int numerator, int denominator)** and **equals(Rational r)** that return **true** if the numerator and denominator values are equal after normalizing.
- A static method **parseRational(char[], char[])** that converts two arrays of numeric characters to a **Rational** object.
- A static method **parseRational(String)** that convert a string containing numeric characters separated by '/' to a **Rational** object.

Draw the UML diagram for the class and then implement the class. Write a client that tests all methods in the class.

*** Source Code / Solution :**

```
package Lab6;

public class Rational {
    private int numerator;
    private int denominator;

    // constructor
    public Rational(int numerator, int denominator) {
        if (denominator == 0) {
            throw new IllegalArgumentException("Denominator cannot be zero.");
        }
        this.numerator = numerator;
        this.denominator = denominator;
    }

    // getters
    public int getNumerator() {
        return numerator;
    }
}
```

```
public int getDenominator() {
    return denominator;
}

// return rational object after reduction of a fraction
public Rational normalize() {
    int gcd = findGCD(numerator, denominator);
    int newDeno = denominator / gcd;
    int newNume = numerator / gcd;

    // put the minus on the numerator
    if (newDeno < 0) {
        newNume *= -1;
        newDeno *= -1;
    }

    return new Rational(newNume, newDeno);
}

// auxiliary method to find the gcd
private static int findGCD(int a, int b) {
    a = Math.abs(a);
    b = Math.abs(b);

    while (b != 0) {
        int tmp = b;
        b = a % b;
        a = tmp;
    }

    return a;
}

// judge whether it is integer
public boolean isWhole() {
    Rational newRational = this.normalize();
    return newRational.getDenominator() == 1;
}

// judge whether it is decimal
public boolean isDecimal() {
    return !isWhole();
}

// judge whether two rationals are equivalent
```

```
    public boolean equals(int numerator, int denominator) {
        Rational newRat = new Rational(numerator,
denominator).normalize();

        Rational thisRat = this.normalize();

        return newRat.numerator == thisRat.numerator &&
newRat.denominator == thisRat.denominator;
    }

    // override: receive the object
    public boolean equals(Rational r) {
        return equals(r.numerator, r.denominator);
    }

    // convert two arrays of numeric characters to a Rational object
    public static Rational parseRational(char[] numerator, char[]
denominator) {
        int nume = Integer.parseInt(new String(numerator));
        int deno = Integer.parseInt(new String(denominator));

        return new Rational(nume, deno);
    }

    // override: receive string seperated by '/' to a Rational object
    public static Rational parseRational(String r) {
        String[] s = r.split("/");
        int nume = Integer.parseInt(s[0].trim());
        int deno = Integer.parseInt(s[1].trim());

        return new Rational(nume, deno);
    }
}
```

RationalTest.java

```
package Lab6;

public class RationalTest {
    public static void main(String[] args) {
        System.out.println("1. Test constructors and normalization");
        Rational r1 = new Rational(4, 8);
        System.out.println("Original: " + r1.getNumerator() + "/" +
r1.getDenominator());
        System.out.println("Normalized: " +
r1.normalize().getNumerator() + "/" + r1.normalize().getDenominator());
    }
}
```

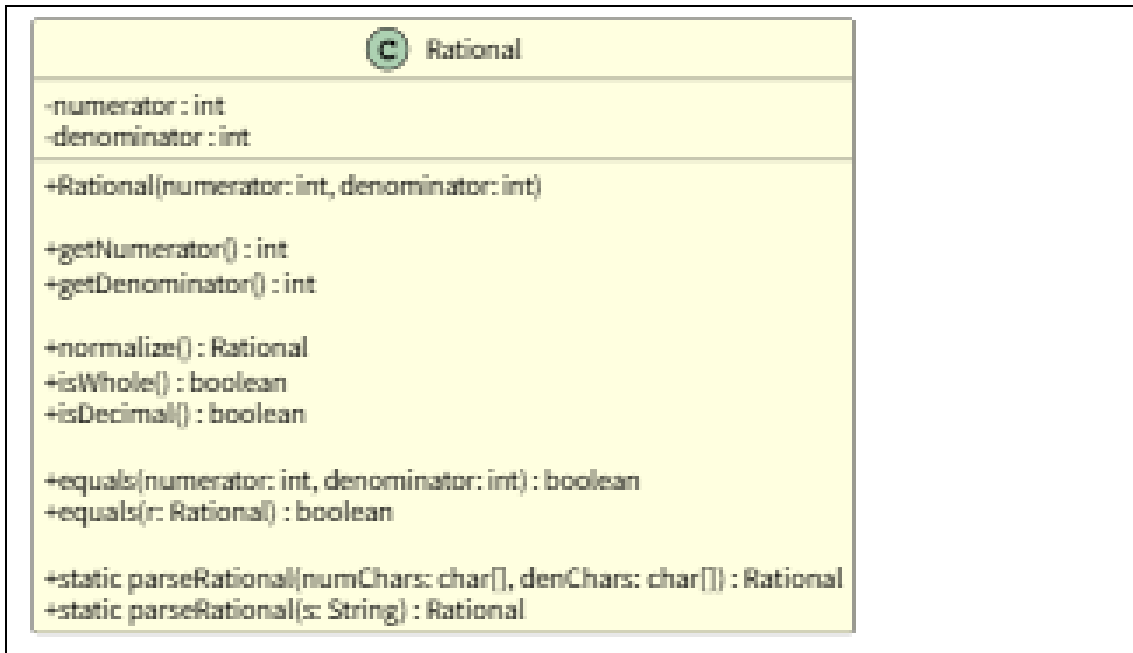
```
// test positive
Rational r2 = new Rational(-2, -3);
System.out.println("Original: " + r2.getNumerator() + "/" +
r2.getDenominator());
System.out.println("Normalized: " +
r2.normalize().getNumerator() + "/" + r2.normalize().getDenominator());

System.out.println("2. Test isWhole() and isDecimal()");
Rational r3 = new Rational(6, 3);
System.out.println(r1.isWhole() ? "r1 is whole." : "r1 is
decimal.");
System.out.println(r3.isDecimal() ? "r3 is decimal." : "r3 is
whole.");

System.out.println("3. Test equal()");
Rational r4 = new Rational(2, 4);
System.out.println(r4.equals(r1) ? "r4 are equal to r1" : "r4
are not equal to r1");
System.out.println(r4.equals(3, 4) ? "r4 are equal to 3/4" :
"r4 are not equal to 3/4");

System.out.println("4. Test parseRational()");
char[] nume = { '1', '2' };
char[] deno = { '6' };
Rational r5 = Rational.parseRational(nume, deno);
System.out.println("r5: " + r5.getNumerator() + '/' +
r5.getDenominator());
Rational r6 = Rational.parseRational("8/31");
System.out.println("r6: " + r6.getNumerator() + '/' +
r6.getDenominator());
    }
}
```

UML graph:



*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab> java Lab6.RationalTest
1. Test constructors and normalization
Original: 4/8
Normalized: 1/2
Original: -2/-3
Normalized: 2/3
2. Test isWhole() and isDecimal()
r1 is decimal.
r3 is whole.
3. Test equal()
r4 are equal to r1
r4 are not equal to 3/4
4. Test parseRational()
r5: 12/6
r6: 8/31
```

*** Debugging/Testing:**

Bug1: Ignore the situation that the denominator and numerator are positive.
Fix: Add the handling of positive numbers in `findGCD()` and `normalize()` to make the result more precise.
Bug2:

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java
Programming\Lab> java Lab6.RationalTest
1. Test constructors and normalization
Original: Lab6.Rational@c387f44
Normalized: Lab6.Rational@73d16e93
Original: Lab6.Rational@659e0bfd
Normalized: Lab6.Rational@2a139a55
...
4. Test parseRational()
Exception in thread "main"
java.lang.NumberFormatException: For input string:
"[C@15db9742"
    at
    java.base/java.lang.NumberFormatException.forInputString(
    NumberFormatException.java:67)
    at
    java.base/java.lang.Integer.parseInt(Integer.java:565)
    at
    java.base/java.lang.Integer.parseInt(Integer.java:662)
    at Lab6.Rational.parseRational(Rational.java:81)
    at Lab6.RationalTest.main(RationalTest.java:27)
Fix: First, directly print object will use default
toString() function, which will return
className@hashCode. So we modify the output format:
    System.out.println("Original: " + r1.getNumerator() + "/" +
    r1.getDenominator());
Then, modify the method when converting
    int nume = Integer.parseInt(new String(numerator));
    int deno = Integer.parseInt(new String(denominator));
```

Problem 3. (10.7)

****10.7** (Game: ATM machine) Use the **Account** class created in Programming Exercise 9.7 to simulate an ATM machine. Create 10 accounts in an array with id **0, 1, . . . , 9**, and an initial balance of \$100. The system prompts the user to enter an id. If the id is entered incorrectly, ask the user to enter a correct id. Once an id is accepted, the main menu is displayed as shown in the sample run. You can enter choice **1** for viewing the current balance, **2** for withdrawing money, **3** for depositing money, and **4** for exiting the main menu. Once you exit, the system will prompt for an id again. Thus, once the system starts, it will not stop.

*** Source Code / Solution :**

```
ATMMachine.java:
package Lab6;

import java.util.Scanner;
```

```
public class ATMMachine {

    public static void main(String[] args) {
        Account[] accounts = new Account[10];
        for (int i = 0; i < 10; i++) {
            accounts[i] = new Account(i, 100);
        }

        Scanner sc = new Scanner(System.in);

        while (true) {
            int id = 0;
            System.out.println("Please enter an id.");
            try {
                id = sc.nextInt();
                if (id < 0 || id >= 10) {
                    System.out.println("Invalid id");
                    continue;
                }
            } catch (java.util.InputMismatchException e) {
                System.out.println("Please enter a correct id.");
                sc.next();
            }
            while (true) {
                int choice = 0;
                System.out.println("Please enter the choice");
                System.out.println("1: check balance");
                System.out.println("2. withdraw money");
                System.out.println("3: deposit money");
                System.out.println("4: exit the main menu");

                try {
                    choice = sc.nextInt();
                    if (choice < 1 || choice > 4) {
                        System.out.println("Invalid choice number");
                        continue;
                    }
                } catch (java.util.InputMismatchException e) {
                    System.out.println("Please enter a correct
choice");
                    sc.next();
                }

                if (choice == 1) {
                    System.out.println("The balance is " +
```



```
accounts[id].getBalance());
        } else if (choice == 2) {
            System.out.println("Enter amount to withdraw");
            double amount = sc.nextDouble();
            accounts[id].withdraw(amount);
            System.out.println("Current balance: " +
accounts[id].getBalance());
        } else if (choice == 3) {
            System.out.println("Enter amount to deposit");
            double amount = sc.nextDouble();
            accounts[id].deposit(amount);
            System.out.println("Current balance: " +
accounts[id].getBalance());
        } else {
            System.out.println("Exiting...");
            break;
        }
    }
}
}
```

Account.java:

```
package Lab6;

import java.util.Date;

public class Account {
    private int id = 0;
    private double balance = 0;
    private double annualInterestRate = 0;
    private Date dateCreated;

    // no-arg constructor
    public Account() {
        id = 0;
        balance = 0;
        annualInterestRate = 0;
        dateCreated = new Date();
    }

    // constructor with specific id and initial balance
    public Account(int id, double balance) {
        this.id = id;
        this.balance = balance;
    }
}
```

```
        this.dateCreated = new Date();
    }

    // accessors
    public int getID() {
        return id;
    }

    public double getBalance() {
        return balance;
    }

    public double getAnnualInterestRate() {
        return annualInterestRate;
    }

    public Date getDateCreated() {
        return dateCreated;
    }

    // mutators
    public void setID(int id) {
        this.id = id;
    }

    public void setBalance(double balance) {
        this.balance = balance;
    }

    public void setAnnualInterestRate(double annualInterestRate) {
        this.annualInterestRate = annualInterestRate;
    }

    // return the monthly interest
    public double getMonthlyInterestRate() {
        return annualInterestRate / 12;
    }

    // return the monthly interest
    public double getMonthlyInterest() {
        double monthlyInterestRate = getMonthlyInterestRate();
        return balance * monthlyInterestRate / 100;
    }

    // withdraws a specified amount, return the account after
```

```
withdrawing
    public double withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            return balance;
        } else {
            throw new IllegalArgumentException("No enough balance");
        }
    }

    // deposit a specified amount, return the account after deposit
    public double deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            return balance;
        } else {
            throw new IllegalArgumentException("Error deposit amount");
        }
    }
}
```

*** Output:**

Jinan University– Java Programming Lab Report

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab> java .\Lab6\ATMMachine.java
Please enter an id.
0
Please enter the choice
1: check balance
2: withdraw money
3: deposit money
4: exit the main menu
1
The balance is 100.0
Please enter the choice
1: check balance
2: withdraw money
3: deposit money
4: exit the main menu
3
Enter amount to deposit
80
Current balance: 180.0
Please enter the choice
1: check balance
2: withdraw money
3: deposit money
4: exit the main menu
2
Enter amount to withdraw
40
Current balance: 140.0
Please enter the choice
1: check balance
2: withdraw money
3: deposit money
4: exit the main menu
9
Invalid choice number
Please enter the choice
1: check balance
2: withdraw money
3: deposit money
Invalid choice number
Please enter the choice
1: check balance
2: withdraw money
3: deposit money
4: exit the main menu
4
Exiting...
Please enter an id.
11
Invalid id
Please enter an id.

```

* Debugging/Testing:

Bug1:

```
System.out.println("Please enter an id.");
try{
    id = sc.nextInt();
} catch (ExceptionInInitializerError e) {
    System.out.println("Please enter a correct id.");
}
```

The code page cannot catch the exception correctly.

Fix: The exception is `java.util.InputMismatchException`. And after receive the error input, clear the buffer.

Bug2:

```
Account[] accounts = new Account[10];
```

```
for (int i = 0; i < 10; i++) {  
    accounts[i].setID(i);  
    accounts[i].setBalance(100);  
}
```

Exception of Null pointer. The array initially is all null pointer. It should be instantiated.

Fix:

```
Account[] accounts = new Account[10];  
for (int i = 0; i < 10; i++) {  
    accounts[i] = new Account(i, 100);  
}
```

Problem 4. (10.9) (Optional)

****10.9** (The **Course** class) Revise the **Course** class as follows:

- Revise the **getStudents()** method to return an array whose length is the same as the number of students in the course. (*Hint*: create a new array and copy students to it.)
- The array size is fixed in Listing 10.6. Revise the **addStudent** method to automatically increase the array size if there is no room to add more students. This is done by creating a new larger array and copying the contents of the current array to it.
- Implement the **dropStudent** method.
- Add a new method named **clear()** that removes all students from the course.

Test your program using https://liveexample.pearsoncmg.com/test/Exercise10_09.txt

*** Source Code / Solution :**

```
package Lab6;  
  
public class Course {  
    private String courseName;  
    private String[] students = new String[100];  
    private int numberOfStudents;  
  
    public Course(String courseName) {  
        this.courseName = courseName;  
    }  
  
    public void addStudent(String student) {  
        int len = students.length;  
        if (numberOfStudents >= len) {  
            String[] newStudents = new String[2 * len];  
            for (int i = 0; i < len; i++) {  
                newStudents[i] = students[i];  
            }  
        }  
    }  
}
```

```
        students = newStudents;
    }

    students[numberOfStudents] = student;
    numberOfStudents++;
}

public String[] getStudents() {
    int length = numberOfStudents;
    String[] students = new String[length];

    for (int i = 0; i < length; i++) {
        students[i] = this.students[i];
    }

    return students;
}

public int getNumberOfStudents() {
    return numberOfStudents;
}

public String getCourseName() {
    return courseName;
}

public void dropStudent(String student) {
    int index = -1;
    for (int i = 0; i < numberOfStudents; i++) {
        if (student.equals(students[i])) {
            index = i;
            break;
        }
    }

    if (index == -1) {
        System.out.println(student + "does not exist");
        return;
    }

    for (int i = index; i < numberOfStudents - 1; i++) {
        students[i] = students[i + 1];
    }

    students[numberOfStudents - 1] = null;
}
```

```
        numberOfStudents--;  
    }  
  
    public void clear() {  
        for (int i = 0; i < numberOfStudents; i++) {  
            students[i] = null;  
        }  
  
        numberOfStudents = 0;  
    }  
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab> java .\Lab6\CourseTest.java  
Number of students in course1: 13  
Peter Jones, Brian Smith, Anne Kennedy, Susan Kennedy, John Kennedy, Kim Johnson, S1, S2, S3, S4, S5, S6, S7,  
Number of students in course2: 2  
Number of students in course1: 12  
Peter Jones, Brian Smith, Anne Kennedy, Susan Kennedy, John Kennedy, Kim Johnson, S2, S3, S4, S5, S6, S7  
Number of students in course1: 0
```

*** Debugging/Testing:**

Bug1: After drop student, the element at students[numberOfStudents - 1] still records the information of last student.

Fix: Let the last element of students[] points at null.

Problem 5. (10.13) (Optional)

***10.13** (Geometry: the **MyRectangle2D** class) Define the **MyRectangle2D** class that contains:

- Two **double** data fields named **x** and **y** that specify the center of the rectangle with getter and setter methods. (Assume the rectangle sides are parallel to **x**- or **y**-axis.)
- The data fields **width** and **height** with getter and setter methods.
- A no-arg constructor that creates a default rectangle with (0, 0) for (x, y) and 1 for both **width** and **height**.
- A constructor that creates a rectangle with the specified **x**, **y**, **width**, and **height**.
- A method **getArea()** that returns the area of the rectangle.
- A method **getPerimeter()** that returns the perimeter of the rectangle.
- A method **contains(double x, double y)** that returns **true** if the specified point (x, y) is inside this rectangle (see Figure 10.24a).
- A method **contains(MyRectangle2D r)** that returns **true** if the specified rectangle is inside this rectangle (see Figure 10.24b).
- A method **overlaps(MyRectangle2D r)** that returns **true** if the specified rectangle overlaps with this rectangle (see Figure 10.24c).



FIGURE 10.24 (a) A point is inside the rectangle. (b) A rectangle is inside another rectangle. (c) A rectangle overlaps another rectangle. (d) Points are enclosed inside a rectangle.

Draw the UML diagram for the class then implement the class. Write a test program that creates a **MyRectangle2D** object **r1** (**new MyRectangle2D(2, 2, 5.5, 4.9)**), displays its area and perimeter, and displays the result of **r1.contains(3, 3)**, **r1.contains(new MyRectangle2D(4, 5, 10.5, 3.2))**, and **r1.overlaps(new MyRectangle2D(3, 5, 2.3, 5.4))**.

*** Source Code / Solution :**

MyRectangle2D.java:

```
package Lab6;

public class MyRectangle2D {
    private double x;
    private double y;
    private double width;
    private double height;

    // getters
    public double getWidth() {
```



```
        return width;
    }

    public double getHeight() {
        return height;
    }

    // setters
    public void setWidth(double width) {
        this.width = width;
    }

    public void setHeight(double height) {
        this.height = height;
    }

    // no-arg constructor
    public MyRectangle2D() {
        x = 0;
        y = 0;
        width = 1;
        height = 1;
    }

    // specified constructor
    public MyRectangle2D(double x, double y, double width, double
height) {
        this.x = x;
        this.y = y;
        this.width = width;
        this.height = height;
    }

    // return the area
    public double getArea() {
        return width * height;
    }

    // return the parimeter
    public double getParimeter() {
        return 2 * (width + height);
    }

    public boolean contains(double x, double y) {
        double dx = width / 2;
```

```
double dy = height / 2;

double thisLeftLimit = x - dx;
double thisRightLimit = x + dx;
double thisDownLimit = y - dy;
double thisUpLimit = y + dy;

return (x < thisRightLimit && x > thisLeftLimit && y <
thisUpLimit + dy && y > thisDownLimit);
}

public boolean contains(MyRectangle2D r) {
    double xRightLimit = r.x + r.width /
        2;
    double xLeftLimit = r.x - r.width / 2;
    double yRightLimit = r.y + r.height / 2;
    double yLeftLimit = r.y - r.height / 2;

    double dx = this.width / 2;
    double dy = this.height / 2;

    double thisLeftLimit = x - dx;
    double thisRightLimit = x + dx;
    double thisDownLimit = y - dy;
    double thisUpLimit = y + dy;

    return (xRightLimit < thisRightLimit && xLeftLimit >
thisLeftLimit && yRightLimit < thisUpLimit
        && yLeftLimit > thisDownLimit);
}

public boolean overlaps(MyRectangle2D r) {
    double xRightLimit = r.x + r.width /
        2;
    double xLeftLimit = r.x - r.width / 2;
    double yRightLimit = r.y + r.height / 2;
    double yLeftLimit = r.y - r.height / 2;

    double dx = this.width / 2;
    double dy = this.height / 2;

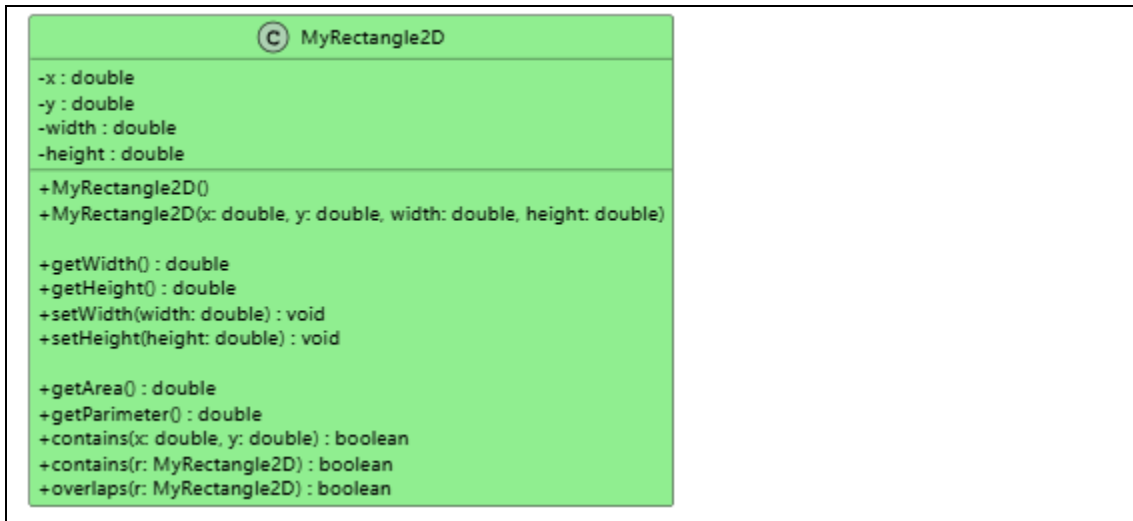
    double thisLeftLimit = x - dx;
    double thisRightLimit = x + dx;
    double thisDownLimit = y - dy;
    double thisUpLimit = y + dy;
```

```
        boolean noOverlap = (thisRightLimit < xLeftLimit) ||  
(thisLeftLimit > xRightLimit) || (thisUpLimit < yLeftLimit)  
        || (thisDownLimit > yRightLimit);  
  
        return !noOverlap;  
    }  
}
```

MyRectangle2DTest.java:

```
package Lab6;  
  
public class MyRectangle2DTest {  
    public static void main(String[] args) {  
        MyRectangle2D r1 = new MyRectangle2D(2, 2, 5.5, 4.9);  
        System.out.println("The area of r1: " + r1.getArea());  
        System.out.println("The parimeter of r1: " +  
r1.getParimeter());  
  
        System.out.println(r1.contains(3, 3)  
            ? "r1 contains (3,3)"  
            : "r1 does not contain (3,3)");  
        System.out.println(r1.contains(new MyRectangle2D(4, 5, 10.5,  
3.2))  
            ? "r1 contains rectangle with 10.5 width and 3.2 height,  
centered (4,5)"  
            : "r1 does not contain rectangle with 10.5 width and  
3.2 height, centered (4,5)");  
  
        System.out.println(r1.overlaps(new MyRectangle2D(3, 5, 2.3,  
5.4))  
            ? "r1 overlaps rectangle with 2.3 width and 5.4 height,  
centered (3,5)"  
            : "r1 does not overlap rectangle with 2.3 width and 5.4  
height, centered (3,5)");  
    }  
}
```

UML graph:



*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab> java .\Lab6\MyRectangle2DTest.java
The area of r1: 26.950000000000003
The parimeter of r1: 20.8
r1 contains (3,3)
r1 does not contain rectangle with 10.5 width and 3.2 height, centered (4,5)
r1 overlaps rectangle with 2.3 width and 5.4 height, centered (3,5)
```

*** Debugging/Testing:**

Bug1 :

```
public boolean overlaps(MyRectangle2D r) {
    double xRightLimit = r.x + r.width /
        2;
    double xLeftLimit = r.x - r.width / 2;
    double yRightLimit = r.y + r.height / 2;
    double yLeftLimit = r.y - r.height / 2;

    boolean isLeftUpIn = contains(xLeftLimit, yRightLimit);
    boolean isLeftDownIn = contains(xLeftLimit, yLeftLimit);
    boolean isRightUpIn = contains(xRightLimit, yRightLimit);
    boolean isRightDownIn = contains(xRightLimit, yLeftLimit);

    return (isLeftDownIn || isLeftUpIn || isRightDownIn ||
isRightUpIn);
}
```

Only judge four corners, ignore the situation that four corners are all not in the rectangle but they still overlap.

Fix: Use the four edge limit to judge.

LAB 7 DATE: 5/16/2026

Student Name: 李荣耀 Student ID: 2024103975**Problem 1. (11.1)**

11.1 (The *Triangle* class) Design a class named **Triangle** that extends **GeometricObject**. The class contains:

- Three **double** data fields named **side1**, **side2**, and **side3** with default values **1.0** to denote three sides of a triangle.
- A no-arg constructor that creates a default triangle.
- A constructor that creates a triangle with the specified **side1**, **side2**, and **side3**.
- The accessor methods for all three data fields.
- A method named **getArea()** that returns the area of this triangle.
- A method named **getPerimeter()** that returns the perimeter of this triangle.
- A method named **toString()** that returns a string description for the triangle.

For the formula to compute the area of a triangle, see Programming Exercise 2.19. The **toString()** method is implemented as follows:

```
return "Triangle: side1 = " + side1 + " side2 = " + side2 +
    " side3 = " + side3;
```

Draw the UML diagrams for the classes **Triangle** and **GeometricObject** and implement the classes. Write a test program that prompts the user to enter three sides of the triangle, a color, and a Boolean value to indicate whether the triangle is filled. The program should create a **Triangle** object with these sides and set the **color** and **filled** properties using the input. The program should display the area, perimeter, color, and true or false to indicate whether it is filled or not.

*** Source Code / Solution :**

Triangle.java:

```
package Lab7;

public class Triangle extends GeometricObject {
    private double side1;
    private double side2;
    private double side3;

    // no-arg constructor
    public Triangle() {
        side1 = 1.0;
        side2 = 1.0;
        side3 = 1.0;
    }

    // constructor with specific side1, side2 and side3
```

```
public Triangle(double side1, double side2, double side3) {
    this.side1 = side1;
    this.side2 = side2;
    this.side3 = side3;
}

// accessor methods
public double getSide1() {
    return side1;
}

public double getSide2() {
    return side2;
}

public double getSide3() {
    return side3;
}

// mutator methods
public void setSide1(double side1) {
    this.side1 = side1;
}

public void setSide2(double side2) {
    this.side2 = side2;
}

public void setSide3(double side3) {
    this.side3 = side3;
}

// return the area of the triangle
public double getArea() {
    double s = (side1 + side2 + side3) / 2.0;
    return Math.sqrt(s * (s - side1) * (s - side2) * (s - side3));
}

// return the perimeter of the triangle
public double getPerimeter() {
    return side1 + side2 + side3;
}

// return a string description of the triangle
public String toString() {
```

```
        return "Triangle: side1 = " + side1 + " side2 = " + side2 + "
side3 = " + side3;
    }
}
```

Triangle Test.java:

```
package Lab7;

import java.util.Scanner;

public class TriangleTest {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Triangle t = new Triangle();

        System.out.println(
            "Please enter three sides, a color and a Boolean value
to indicate whether the triangle is filled.");

        double side1 = sc.nextDouble();
        double side2 = sc.nextDouble();
        double side3 = sc.nextDouble();

        if (side1 <= 0 || side2 <= 0 || side3 <= 0) {
            System.out.println("Invalid input. Please enter positive
values for all sides.");
            sc.close();
            return;
        }

        String color = sc.next();
        boolean filled = sc.nextBoolean();

        t.setSide1(side1);
        t.setSide2(side2);
        t.setSide3(side3);
        t.setColor(color);
        t.setFilled(filled);

        System.out.println("Area: " + t.getArea());
        System.out.println("Perimeter: " + t.getPerimeter());
        System.out.println("Color: " + t.getColor());
        System.out.println("Filled: " + t.isFilled());

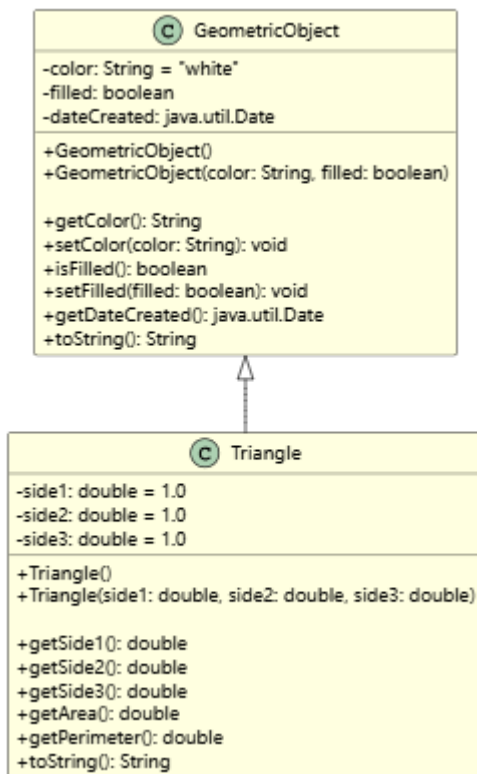
        sc.close();
    }
}
```

```
}  
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programmin  
g\Lab> java .\Lab7\TriangleTest.java  
Please enter three sides, a color and a Boolean value to indi  
cate whether the triangle is filled.  
3 4 5 green true  
Area: 6.0  
Perimeter: 12.0  
Color: green  
Filled: true
```

UML Graph:



*** Debugging/Testing:**

Bug1: The requests do not include the mutators in the class **Triangle**, which might affect receiving arguments.

Fix: Add three mutator methods for three sides.

Bug2: The area may encounter negative values.

Fix: Limit the sides when entering the arguments.

Problem 2. (11.5)

11.5 (The **Course** class) Rewrite the **Course** class in Listing 10.6. Use an **ArrayList** to replace an array to store students. Draw the new UML diagram for the class. You should not change the original contract of the **Course** class (i.e., the definition of the constructors and methods should not be changed, but the private members may be changed.)

*** Source Code / Solution :**

Course.java:

```
package Lab7;

import java.util.ArrayList;

public class Course {
    private String courseName;
    private ArrayList<String> students = new ArrayList<String>();
    private int numberOfStudents;

    public Course(String courseName) {
        this.courseName = courseName;
    }

    public void addStudent(String student) {
        students.add(student);
        numberOfStudents++;
    }

    public ArrayList<String> getStudents() {
        return students;
    }

    public int getNumberOfStudents() {
        return numberOfStudents;
    }

    public String getCourseName() {
        return courseName;
    }

    public void dropStudent(String student) {
        students.remove(student);
        numberOfStudents--;
    }
}
```

```
}
```

CourseTest.java

```
package Lab7;

import java.util.ArrayList;

public class CourseTest {
    public static void main(String[] args) {
        Course course1 = new Course("Data Structures");
        Course course2 = new Course("Database Systems");

        course1.addStudent("Peter Jones");
        course1.addStudent("Brian Smith");
        course1.addStudent("Anne Kennedy");
        course1.addStudent("Susan Kennedy");
        course1.addStudent("John Kennedy");
        course1.addStudent("Kim Johnson");
        course1.addStudent("S1");
        course1.addStudent("S2");
        course1.addStudent("S3");
        course1.addStudent("S4");
        course1.addStudent("S5");
        course1.addStudent("S6");
        course1.addStudent("S7");

        course2.addStudent("Peter Jones");
        course2.addStudent("Steve Smith");

        System.out.println("Number of students in course1: "
            + course1.getNumberOfStudents());
        ArrayList<String> students = course1.getStudents();
        for (int i = 0; i < students.size(); i++)
            System.out.print(students.get(i) + ", ");

        System.out.println();
        System.out.println("Number of students in course2: "
            + course2.getNumberOfStudents());

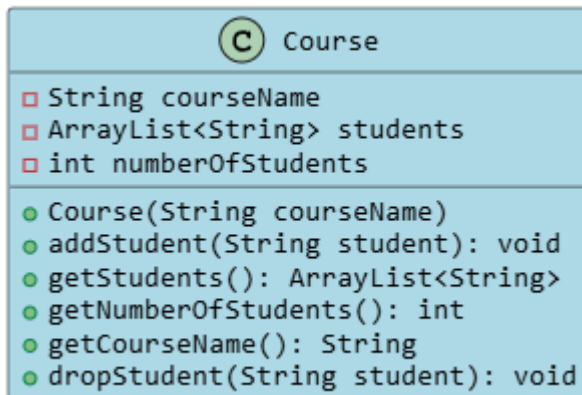
        course1.dropStudent("S1");
        System.out.println("Number of students in course1: "
            + course1.getNumberOfStudents());
        students = course1.getStudents();
        for (int i = 0; i < course1.getNumberOfStudents(); i++)
            System.out.print(students.get(i) + (i <
```

```
course1.getNumberOfStudents() - 1 ? ", " : " ");  
  
    }  
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab> java .\Lab7\CourseTest.java  
Number of students in course1: 13  
Peter Jones, Brian Smith, Anne Kennedy, Susan Kennedy, John Kennedy, Kim Johnson, S1,  
S2, S3, S4, S5, S6, S7,  
Number of students in course2: 2  
Number of students in course1: 12  
Peter Jones, Brian Smith, Anne Kennedy, Susan Kennedy, John Kennedy, Kim Johnson, S2,  
S3, S4, S5, S6, S7
```

UML Graph:



*** Debugging/Testing:**

Bug1: When compiling and running CourseTest.java, it cannot specify the ArrayList type.

Fix: Add the import statement in CourseTest.java. In the same package, the class can directly use but the package imported in one class will not be shared in other class.

Problem 3. (11.11)

11.11 (Sort **ArrayList**) Write the following method that sorts an **ArrayList** of numbers:

```
public static void sort(ArrayList<Integer> list)
```

Write a test program that prompts the user to enter five numbers, stores them in an array list, and displays them in increasing order.

11.12 (Sum **ArrayList**) Write the following method that returns the sum of all num

*** Source Code / Solution :**

```
package Lab7;
```

```
import java.util.ArrayList;
import java.util.Scanner;

public class SortArrayList {
    public static void sort(ArrayList<Integer> list) {
        // sort the list by increasing order
        list.sort(null);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<Integer> numbers = new ArrayList<Integer>();

        System.out.println("Please enter five numbers, separated by
space:");
        int cnt = 5;
        while (cnt > 0) {
            numbers.add(sc.nextInt());
            cnt--;
        }

        sort(numbers);

        System.out.println("Sorted list:");
        for (int num : numbers) {
            System.out.print(num + " ");
        }

        System.out.println();

        sc.close();
    }
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab> java .\Lab7\SortAr
rayList.java
Please enter five numbers, separated by space:
8
1
3
5
6
Sorted list:
1 3 5 6 8
```

*** Debugging/Testing:**

Bug1: Syntax error when creating objects in type of ArrayList.

Fix:

```
ArrayList<Integer> numbers = new ArrayList<Integer>();
```

ArrayList<E>, the type can only be the reference type. We should use the corresponding reference type if using base type.

And we use the method in the ArrayList class to make the code clear and efficient.

Problem 4. (11.13) (Optional)

***11.13** (Remove duplicates) Write a method that removes the duplicate elements from an array list of integers using the following header:

```
public static void removeDuplicate(ArrayList<Integer> list)
```

Write a test program that prompts the user to enter 10 integers to a list and displays the distinct integers in their input order and separated by exactly one space. Here is a sample run:

```
Enter 10 integers: 34 5 3 5 6 4 33 2 2 4 Enter
The distinct integers are 34 5 3 6 4 33 2
```

*** Source Code / Solution :**

```
package Lab7;

import java.util.Scanner;
import java.util.ArrayList;

public class RemoveDuplicates {
    public static void removeDuplicate(ArrayList<Integer> list) {
        int len = list.size();
        for (int i = 0; i < len; i++) {
            for (int j = i + 1; j < len; j++) {
```

```
        if (list.get(i) == list.get(j)) {
            list.remove(j);
            len--;
        }
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    ArrayList<Integer> list = new ArrayList<Integer>();
    System.out.println("Enter 10 integers: ");
    int len = 10;
    for (int i = 0; i < len; i++) {
        list.add(sc.nextInt());
    }

    removeDuplicate(list);
    len = list.size();
    System.out.println("The distinct integers are: ");
    for (int i = 0; i < len; i++) {
        System.out.print(list.get(i) + " ");
    }

    sc.close();
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab> java .\Lab7\Remove
Duplicates.java
Enter 10 integers:
2 2 3 4 7 8 9 8 1 3
The distinct integers are:
2 3 4 7 8 9 1
```

*** Debugging/Testing:**

Bug1: The iterator is out of the bound of the array.

Fix:

```
        if (list.get(i) == list.get(j)) {
            list.remove(j);
            len--;
        }
```

Add the len-- in the method to correct the logic.

Problem 5. (11.17) (Optional)

****11.17** (Algebra: perfect square) Write a program that prompts the user to enter an integer m and find the smallest integer n such that $m * n$ is a perfect square. (Hint: Store all smallest factors of m into an array list. n is the product of the factors that appear an odd number of times in the array list. For example, consider $m = 90$, store the factors 2, 3, 3, and 5 in an array list. 2 and 5 appear an odd number of times in the array list. Thus, n is 10.) Here is a sample run of the program:

> Enter an integer m: 1500
The smallest number n for m * n to be a perfect square is 15
m * n is 22500

> Enter an integer m: 63
The smallest number n for m * n to be a perfect square is 7
m * n is 441

*** Source Code / Solution :**

```
package Lab7;

import java.util.ArrayList;
import java.util.Scanner;

public class PerfectSquare {
    public static int findPerfectSquare(int m) {
        ArrayList<Integer> factors = new ArrayList<Integer>();

        // store all smallest factors of m in factors array list
        for (int i = 2; i * i <= m; i++) {
            while (m % i == 0) {
                factors.add(i);
                m /= i;
            }
        }

        if (m > 1) {
            factors.add(m);
        }

        // find all factors appearing in odd numbers
        factors.sort(null);
        int len = factors.size();
        int n = 1;
    }
}
```

```
int cnt = 1;
for (int i = 0; i < len; i++) {
    if (i + 1 < len && factors.get(i) == factors.get(i + 1)) {
        cnt++;
    } else {
        if (cnt % 2 == 1) {
            n *= factors.get(i);
        }

        cnt = 1;
    }
}

return n;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.println("Enter an integer m: ");
    int m = sc.nextInt();
    int n = findPerfectSquare(m);
    System.out.println("The smallest number n for m * n to be a
perfect square is " + n);
    System.out.println("m * n is " + m * n);

    sc.close();
}
}
```

*** Output:**


```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab> java .\Lab7\PerfectSquare.java
Enter an integer m:
1500
The smallest number n for m * n to be a perfect square is 15
m * n is 22500
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab> java .\Lab7\PerfectSquare.java
Enter an integer m:
63
The smallest number n for m * n to be a perfect square is 7
m * n is 441
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab> java .\Lab7\PerfectSquare.java
Enter an integer m:
10
The smallest number n for m * n to be a perfect square is 10
m * n is 100
```

*** Debugging/Testing:**

Bug1: The result is incorrect for n.

Fix:

```
while (m % i == 0) {
    factors.add(i);
    m /= i;
}
```

Correct the logic that use loop rather than one judgment.

LAB 8 DATE: 5/19/2026

Student Name: 李荣耀 **Student ID:** 2024103975

Problem 1. (12.3)

*** 12.3** (*InputMismatchException*) The previous program works well as long as the user enters an integer. Otherwise, you may get another kind of exception. For instance, if you use `nextInt()` of Scanner, you could have an *InputMismatchException*. Modify it to prevent users entering anything other than an integer.

*** Source Code / Solution :**

```
package Lab8;

import java.util.InputMismatchException;
import java.util.Scanner;

public class MonthAndDays {
    public static void main(String[] args) {
```

```
String[] months = { "January", "February", "March", "April",  
    "May", "June", "July", "August", "September",  
    "October",  
    "November", "December" };  
int[] dom = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };  
Scanner sc = new Scanner(System.in);  
int idx;  
String month;  
int days;  
  
System.out.println("Please enter an integer between 1 and 12:  
");  
  
try{  
    idx = sc.nextInt() - 1;  
} catch (InputMismatchException e) {  
    System.err.println("Invalid input. Please enter an integer  
between 1 and 12.");  
    sc.close();  
    return;  
}  
  
try{  
    month = months[idx];  
    days = dom[idx];  
} catch (ArrayIndexOutOfBoundsException e) {  
    System.err.println("Wrong number. Please enter an integer  
between 1 and 12.");  
    sc.close();  
    return;  
}  
  
System.out.println("Month : " + month);  
System.out.println("The number of days in the month: " + days);  
  
sc.close();  
}  
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab> java .\Lab8\MonthAndDays.java
Please enter an integer between 1 and 12:
3
Month :March
The number of days in the month: 31
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab> java .\Lab8\MonthAndDays.java
Please enter an integer between 1 and 12:
i
Invalid input. Please enter an integer between 1 and 12.
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab> java .\Lab8\MonthAndDays.java
Please enter an integer between 1 and 12:
19
Wrong number. Please enter an integer between 1 and 12.
```

*** Debugging/Testing:**

Bug1: The variables defined in the try block cannot be used outside.

Fix: Define the variable outside the try_catch block. The domain of those variables defined in the block does not include the outside.

Problem 2. (12.7)

***12.7 (Checked Exception)** In preparation for a class **Fraction**, create a class **NullDenominatorException** that must be a checked exception whose instances are thrown if the denominator of an object of class **Fraction** is 0.

*** Source Code / Solution :**

```
Fraction:
package Lab8;

public class Fraction {
    private int numerator;
    private int denominator;

    public Fraction(int numerator, int denominator) throws
    NullDenominatorException {
        if (denominator == 0) {
            throw new NullDenominatorException("The denominator cannot
            be zero.");
        }
    }
}
```

```
        this.numerator = numerator;
        this.denominator = denominator;
    }

    // accessor methods
    public int getNumerator() {
        return numerator;
    }

    public int getDenominator() {
        return denominator;
    }

    // mutator methods
    public void setNumerator(int numerator) {
        this.numerator = numerator;
    }

    public void setDenominator(int denominator) throws
    NullDenominatorException {
        if (denominator == 0){
            throw new NullDenominatorException("The denominator cannot
            be zero.");
        }

        this.denominator = denominator;
    }

    public class NullDenominatorException extends Exception {
        public NullDenominatorException() {
            super();
        }

        public NullDenominatorException(String message) {
            super(message);
        }
    }
}
```

FractionTest:

```
package Lab8;
```

```
import Lab8.Fraction.NullDenominatorException;

public class FractionTest {
    public static void main(String[] args) {
        try{
            Fraction f = new Fraction(2, 0);
        } catch (NullDenominatorException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab> java .\Lab8\FractionTest.java
The denominator cannot be zero.
```

*** Debugging/Testing:**

Bug1: After creating the class NullDenominatorException and invoking by

```
        if (denominator == 0) {
            throw new NullDenominatorException("The denominator cannot be zero.");
        }
```

The compiler reports error when compiling.

Fix: If a method throw out exceptions, there must be throws statement in the method signature. So state throws when constructing and setting.

Bug2: In test program, it occurs the same situation.

Fix: The mechanism of throw will find the first matching catch until main function. If no catch the program will crash.

Problem 3. (12.11)

****12.11** (Remove text) Write a program that removes all the occurrences of a specified string from a text file. For example, invoking

```
java Exercise12_11 John filename
```

removes the string **John** from the specified file. Your program should get the arguments from the command line.

*** Source Code / Solution :**

```
package Lab8;
```

```
import java.io.FileNotFoundException;
import java.io.IOException;
import java.nio.file.*;

public class RemoveText {
    public static void main(String[] args) {
        if (args.length < 2) {
            System.out.println("Please enter: java RemoveText <target>
<filename>.");
            return;
        }

        String target = args[0];
        String filename = args[1];

        try {
            // read the file
            Path path = Paths.get(filename);
            String content = new String(Files.readAllBytes(path));

            // remove the target text
            String newContent = content.replace(target, "");

            // rewrite the modified file
            Files.write(path, newContent.getBytes());

            System.out.println("Finished.");
        } catch (FileNotFoundException e) {
            System.err.println("Not Found the file " + filename);
        } catch (IOException e) {
            System.err.println("Error when reading and writing the
file. " + e.getMessage());
        }
    }
}
```

*** Output:**

```
test.txt:
```

Lab8 >  test.txt

```
1   There are places I'll remember
2   All my life, though some have changed
3   Some forever, not for better
4   Some have gone and some remain
5   All these places have their moments
6   With lovers and friends I still can recall
7   Some are dead and some are living
8   In my life I've loved them all
9   But of all these friends and lovers
10  There is no one compares with you
11  And these memories lose their meaning
12  When I think of love as something new
13  Though I know I'll never lose affection
14  For people and things that went before
15  I know I'll often stop and think about them
16  In my life, I love you more
17  Though I know I'll never lose affection
18  For people and things that went before
19  I know I'll often stop and think about them
20  In my life, I love you more
21  In my life, I love you more
22
```

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programmin
g\Lab\Lab8> java RemoveText.java
Please enter: java RemoveText <target> <filename>.
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programmin
g\Lab\Lab8> java RemoveText.java life test.txt
Finished.
```

After running:

Lab8 > test.txt

```
1  There are places I'll remember
2  All my , though some have changed
3  Some forever, not for better
4  Some have gone and some remain
5  All these places have their moments
6  With lovers and friends I still can recall
7  Some are dead and some are living
8  In my I've loved them all
9  But of all these friends and lovers
10 There is no one compares with you
11 And these memories lose their meaning
12 When I think of love as something new
13 Though I know I'll never lose affection
14 For people and things that went before
15 I know I'll often stop and think about them
16 In my , I love you more
17 Though I know I'll never lose affection
18 For people and things that went before
19 I know I'll often stop and think about them
20 In my , I love you more
21 In my , I love you more
```

*** Debugging/Testing:**

Bug1: No exception handling when the arguments in the command line is less than 2.

Fix:

```
    if (args.length < 2) {
        System.out.println("Please enter: java RemoveText <target>
<filename>.");
        return;
    }
```

Problem 4. (12.27) (Optional)

****12.27** (Replace words) Suppose you have a lot of files in a directory that contain words **Exercise_i__j**, where *i* and *j* are digits. Write a program that pads a 0 before *i* if *i* is a single digit and 0 before *j* if *j* is a single digit. For example, the word **Exercise2_1** in a file will be replaced by **Exercise02_01**. In Java, when you pass the symbol * from the command line, it refers to all files in the directory (see Supplement III.V). Use the following command to run your program:

```
java Exercise12_27 *
```

*** Source Code / Solution :**

```
package Lab8;

import java.io.FileNotFoundException;
import java.io.IOException;
import java.nio.file.*;
import java.util.regex.Pattern;
import java.util.regex.Matcher;

public class ReplaceWords {
    public static void main(String[] args) {
        if (args.length < 1) {
            System.out.println("Usage: java RenameFiles *");
            return;
        }

        Pattern pattern = Pattern.compile("Exercise(\\d+)_ (\\d+)");

        for (String filename : args) {
            Path path = Paths.get(filename);
            if (Files.isDirectory(path)) {
                continue;
            }

            try {
                String content = new String(Files.readAllBytes(path));
                StringBuffer sb = new StringBuffer();
                Matcher matcher = pattern.matcher(content);

                // replace words for all matches
                while (matcher.find()) {
                    String i = matcher.group(1);
                    String j = matcher.group(2);
                    // pad a 0 before single digit numbers
                    String paddedI = i.length() == 1 ? "0" + i : i;
                    String paddedJ = j.length() == 1 ? "0" + j : j;
                }
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    }
}
```

```
        String newWord = "Exercise" + paddedI + "_" +
paddedJ;
        matcher.appendReplacement(sb, newWord);
    }

    matcher.appendTail(sb);
    String newContent = sb.toString();

    if (!newContent.equals(content)) {
        Files.write(path, newContent.getBytes());
        System.out.println("Updated :" + filename);
    }
} catch (FileNotFoundException e) {
    System.err.println("Not Found the file " + filename);
} catch (IOException e) {
    System.err.println("Error when handling file: " +
filename + " - " + e.getMessage());
}
}
}
```

*** Output:**

ReplaceWordsTest1.txt:

Lab8 > ReplaceWordsTest >  ReplaceWordsTest1.txt

```
1   Exercise1_2
2   Exercise1_3
3   Exercise01_4
4   Exercise10_5
5   Exercise12_23
6   Exercise12_4
```

ReplaceWordsTest2.txt:

```
Lab8 > ReplaceWordsTest > ReplaceWordsTest2.txt
```

```
1 Exercise2_5
2 Not found
3 Not target
4 Exercise2_6
5 Not found
6 Exercise3_10
7 Not target
8 Exercise3_11
```

```
t2.txt java .\Lab8\ReplaceWords.java .\Lab8\ReplaceWordsTest\
ReplaceWordsTest1.txt .\Lab8\ReplaceWordsTest\ReplaceWordsTes
t2.txt
```

```
Updated :.\Lab8\ReplaceWordsTest\ReplaceWordsTest1.txt
```

```
Updated :.\Lab8\ReplaceWordsTest\ReplaceWordsTest2.txt
```

After running:

ReplaceWordsTest1.txt:

```
Lab8 > ReplaceWordsTest > ReplaceWordsTest1.txt
```

```
1 Exercise01_02
2 Exercise01_03
3 Exercise01_04
4 Exercise10_05
5 Exercise12_23
6 Exercise12_04
```

ReplaceWordsTest2.txt:

```
Lab8 > ReplaceWordsTest > ReplaceWordsTest2.txt
```

```
1 Exercise02_05
2 Not found
3 Not target
4 Exercise02_06
5 Not found
6 Exercise03_10
7 Not target
8 Exercise03_11
```

*** Debugging/Testing:**

Bug1: Unreachable catch block for FileNotFoundException.

Fix: Modify the order of IOException and FileNotFoundException.

Bug2: Incorrect input in the command line.

Fix: In Windows powershell and cmd, the wildcard character cannot be ignored correctly sometimes. So I have to enter all file names.

Problem 5. (12.33) (Optional)

****12.33** (Search Web) Modify Listing 12.18 WebCrawler.java to search for the word (e.g., Computer Programming) starting from a URL (e.g., <http://cs.armstrong.edu/liang>). Your program prompts the user to enter the word and the starting URL and terminates once the word is found. Display the URL for the page that contains the word.

*** Source Code / Solution :**

```
package Lab8;

import java.util.Scanner;
import java.util.ArrayList;

public class WebCrawler {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter a word to search: ");
        String word = input.nextLine();
        System.out.print("Enter a URL: ");
        String url = input.nextLine();
        crawler(url, word); // Traverse the Web from the starting url
    }

    public static void crawler(String startingURL, String word) {
        ArrayList<String> listOfPendingURLs = new ArrayList<>();
        ArrayList<String> listOfTraversedURLs = new ArrayList<>();

        listOfPendingURLs.add(startingURL);
        while (!listOfPendingURLs.isEmpty() &&
            listOfTraversedURLs.size() <= 100) {
            String urlString = listOfPendingURLs.remove(0);
            if (!listOfTraversedURLs.contains(urlString)) {
                listOfTraversedURLs.add(urlString);
                System.out.println("Crawl " + urlString);
            }
        }
    }
}
```

```
        // check whether current page includes searching word
        if (pageContainsWord(urlString, word)) {
            System.out.println("Word found at: " + urlString);
            return;
        }

        for (String s : getSubURLs(urlString)) {
            if (!listOfTraversedURLs.contains(s))
                listOfPendingURLs.add(s);
        }
    }
    System.out.println("Word not found within the crawled pages.");
}

public static boolean pageContainsWord(String urlString, String
word) {
    try {
        java.net.URL url = new java.net.URL(urlString);
        Scanner input = new Scanner(url.openStream());

        StringBuilder pageContent = new StringBuilder();
        while (input.hasNextLine()) {
            pageContent.append(input.nextLine()).append("\n");
        }
        input.close();

        return pageContent.toString().toLowerCase()
            .contains(word.toLowerCase());
    } catch (Exception ex) {
        System.out.println("Error reading page: " +
ex.getMessage());
        return false;
    }
}

public static ArrayList<String> getSubURLs(String urlString) {
    ArrayList<String> list = new ArrayList<>();

    try {
        java.net.URL url = new java.net.URL(urlString);
        Scanner input = new Scanner(url.openStream());
        int current = 0;
        while (input.hasNext()) {
```

```
String line = input.nextLine();
current = line.indexOf("http:", current);
while (current > 0) {
    int endIndex = line.indexOf("\"", current);
    if (endIndex > 0) { // Ensure that a correct URL is
found
        list.add(line.substring(current, endIndex));
        current = line.indexOf("http:", endIndex);
    } else
        current = -1;
    }
}
} catch (Exception ex) {
    System.out.println("Error: " + ex.getMessage());
}

return list;
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab> java .\Lab8\WebCra
wler.java
.\Lab8\WebCrawler.java:45: 警告: [deprecation] URL中的URL(String)已过时
    java.net.URL url = new java.net.URL(urlString);
                        ^
.\Lab8\WebCrawler.java:66: 警告: [deprecation] URL中的URL(String)已过时
    java.net.URL url = new java.net.URL(urlString);
                        ^
2 个警告
Enter a word to search: Java programming
Enter a URL: https://media.pearsoncmg.com/ph/esm/ecs_liang_11/cw/
Crawl https://media.pearsoncmg.com/ph/esm/ecs_liang_11/cw/
Word found at: https://media.pearsoncmg.com/ph/esm/ecs_liang_11/cw/
```

*** Debugging/Testing:**

Bug1: The example web URL cannot be accessed.

Fix: The URL cannot be accessed due to the cause of region. So, I replace it by another web URL.

Problem 6. (13.1)

****13.1** (Triangle class) Design a new **Triangle** class that extends the abstract **GeometricObject** class. Draw the UML diagram for the classes **Triangle** and **GeometricObject** then implement the **Triangle** class. Write a test program that prompts the user to enter three sides of the triangle, a color, and a Boolean value to indicate whether the triangle is filled. The program should create a **Triangle** object with these sides, and set the **color** and **filled** properties using the input. The program should display the area, perimeter, color, and true or false to indicate whether it is filled or not.

*** Source Code / Solution :**

Triangle.java:

```
package Lab8;

public class Triangle extends GeometricObject{
    private double side1;
    private double side2;
    private double side3;

    //accessor methods
    public double getSide1() {
        return side1;
    }

    public double getSide2() {
        return side2;
    }

    public double getSide3() {
        return side3;
    }

    // mutator methods
    public void setSide1(double side1) {
        this.side1 = side1;
    }

    public void setSide2(double side2) {
        this.side2 = side2;
    }

    public void setSide3(double side3) {
        this.side3 = side3;
    }

    //constructor
```

```
public Triangle() {
    this(1.0, 1.0, 1.0);
}

public Triangle(double side1, double side2, double side3) {
    this.side1 = side1;
    this.side2 = side2;
    this.side3 = side3;
}

public Triangle(double side1, double side2, double side3, String
color, boolean filled) {
    super(color, filled);
    this.side1 = side1;
    this.side2 = side2;
    this.side3 = side3;
}

@Override
public double getArea() {
    double s = (side1 + side2 + side3) / 2;
    return Math.sqrt(s * (s - side1) * (s - side2) * (s - side3));
}

@Override
public double getPerimeter() {
    return side1 + side2 + side3;
}

@Override
public String toString() {
    return "Triangle: side1 = " + side1 + ", side2 = " + side2 + ",
side3 = " + side3 + "\n" + super.toString();
}
}
```

TriangleTest.java:

```
package Lab8;

import java.util.Scanner;

public class TriangleTest {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double side1 = 1.0;
```



```
double side2 = 1.0;
double side3 = 1.0;
String color = "white";
boolean filled = false;

try{
    System.out.println("Enter three sides of the triangle: ");
    side1 = sc.nextDouble();
    side2 = sc.nextDouble();
    side3 = sc.nextDouble();

    System.out.println("Enter a color: ");
    color = sc.next();

    System.out.println("Enter filled or not (true/false): ");
    filled = sc.nextBoolean();
} catch (Exception e) {
    System.err.println("Invalid input format!");
    sc.close();
    return;
}
Triangle t = new Triangle(side1, side2, side3, color, filled);

System.out.println("Area of the triangle is: " + t.getArea());
System.out.println("Perimeter of the triangle is: " +
t.getPerimeter());
System.out.println("Color of the triangle is: " +
t.getColor());
System.out.println("Filled or not: " + t.isFilled());

sc.close();
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\Lab8> java .\TriangleTest.java
Enter three sides of the triangle:
3 4 5
Enter a color:
red
Enter filled or not (true/false):
true
Area of the triangle is: 6.0
Perimeter of the triangle is: 12.0
Color of the triangle is: red
Filled or not: true
```

*** Debugging/Testing:**

Bug1: No Exception handling when entering incorrect format.

Fix:

```
try{
    ...
} catch (Exception e) {
    System.err.println("Invalid input format!");
    sc.close();
    return;
}
```

Problem 7. (13.7)

***13.7** (The **Colorable** interface) Design an interface named **Colorable** with a **void** method named **howToColor()**. Every class of a colorable object must implement the **Colorable** interface. Design a class named **Square** that extends **GeometricObject** and implements **Colorable**. Implement **howToColor** to display the message **Color all four sides**. The **Square** class contains a data field **side** with getter and setter methods, and a constructor for constructing a **Square** with a specified side. The **Square** class has a private double data field named **side** with its getter and setter methods. It has a no-arg constructor to create a **Square** with side 0, and another constructor that creates a **Square** with the specified side.

Draw a UML diagram that involves **Colorable**, **Square**, and **GeometricObject**. Write a test program that creates an array of five **GeometricObjects**. For each object in the array, display its area and invoke its **howToColor** method if it is colorable.

*** Source Code / Solution :**

Colorable.java:

```
package Lab8;
```

```
public interface Colorable {  
    void howToColor();  
}
```

Square.java

```
package Lab8;  
  
public class Square extends GeometricObject implements Colorable {  
    private double side;  
  
    // constructor  
    public Square() {  
        this(0);  
    }  
  
    public Square(double side) {  
        super();  
        this.side = side;  
    }  
  
    // setter  
    public void setSide(double side) {  
        this.side = side;  
    }  
  
    // getter  
    public double getSide() {  
        return side;  
    }  
  
    @Override  
    public void howToColor() {  
        System.out.println("Color all four sides");  
    }  
  
    public double getArea() {  
        return side * side;  
    }  
  
    public double getPerimeter() {  
        return 4 * side;  
    }  
  
    @Override  
    public String toString() {  
        return "Square: side = " + side + ", area = " + getArea() + ",  
    }  
}
```

```
perimeter = " + getPerimeter()  
            + super.toString();  
    }  
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\Lab8> java .\SquareTest.java  
Enter the type of object (square/triangle):  
square  
Enter the side of the square:  
3  
Enter the color of the square:  
red  
Enter whether the square is filled or not (true/false):  
true  
Enter the type of object (square/triangle):  
triangle  
Enter three sides of the triangle:  
2 3 4  
Enter the color of the triangle:  
blue  
Enter whether the triangle is filled or not (true/false):  
false  
Enter the type of object (square/triangle):  
square  
Enter the side of the square:  
2  
Enter the color of the square:  
blue  
Enter whether the square is filled or not (true/false):  
false  
Enter the type of object (square/triangle):  
square  
Enter the side of the square:  
3  
Enter the color of the square:  
black  
Enter whether the square is filled or not (true/false):  
false  
Enter the type of object (square/triangle):  
triangle  
Enter three sides of the triangle:  
3 4 5  
Enter the color of the triangle:  
white
```

```

false
Enter the type of object (square/triangle):
triangle
Enter three sides of the triangle:
3 4 5
Enter the color of the triangle:
white
Enter whether the triangle is filled or not (true/false):
false
Square: side = 3.0, area = 9.0, perimeter = 12.0
created on Wed May 20 15:57:55 CST 2026
color: red and filled: true
How to color the object:
Color all four sides

Triangle: side1 = 2.0, side2 = 3.0, side3 = 4.0
created on Wed May 20 15:58:09 CST 2026
color: blue and filled: false

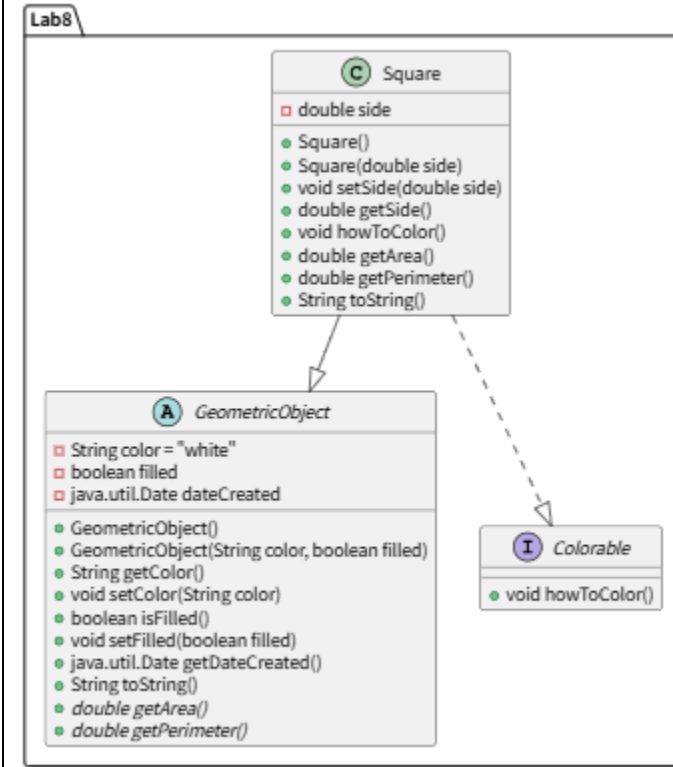
Square: side = 2.0, area = 4.0, perimeter = 8.0
created on Wed May 20 15:58:20 CST 2026
color: blue and filled: false
How to color the object:
Color all four sides

Square: side = 3.0, area = 9.0, perimeter = 12.0
created on Wed May 20 15:58:32 CST 2026
color: black and filled: false
How to color the object:
Color all four sides

Triangle: side1 = 3.0, side2 = 4.0, side3 = 5.0
created on Wed May 20 15:58:44 CST 2026
color: white and filled: false
    
```

PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programming\Lab\Lab8> █

UML graph:



*** Debugging/Testing:**

Bug1: The for statement cannot be recognized by the compiler.

Fix: Forget the main function in the test program.

Bug2: Sometimes the type input will be jumped over.

Fix: The nextline character is stored in the buffer and will be read by `sc.nextline()`. So we add `sc.nextline()` after other reading.

Problem 8. (13.17) (Optional)

***13.17** (Math: The **Complex** class) A complex number is a number in the form $a + bi$, where a and b are real numbers and i is $\sqrt{-1}$. The numbers a and b are known as the real part and imaginary part of the complex number, respectively. You can perform addition, subtraction, multiplication, and division for complex numbers using the following formulas:

$$a + bi + c + di = (a + c) + (b + d)i$$

$$a + bi - (c + di) = (a - c) + (b - d)i$$

$$(a + bi) * (c + di) = (ac - bd) + (bc + ad)i$$

$$(a + bi)/(c + di) = (ac + bd)/(c^2 + d^2) + (bc - ad)i/(c^2 + d^2)$$

You can also obtain the absolute value for a complex number using the following formula:

$$|a + bi| = \sqrt{a^2 + b^2}$$

(A complex number can be interpreted as a point on a plane by identifying the (a, b) values as the coordinates of the point. The absolute value of the complex number corresponds to the distance of the point to the origin, as shown in Figure 13.10.)

Design a class named **Complex** for representing complex numbers and the methods **add**, **subtract**, **multiply**, **divide**, and **abs** for performing complex-number operations, and override **toString** method for returning a string representation for a complex number. The **toString** method returns $(a + bi)$ as a string. If b is **0**, it simply returns a . Your **Complex** class should also implement **Cloneable** and **Comparable**. Compare two complex numbers using their absolute values.

Provide three constructors **Complex(a, b)**, **Complex(a)**, and **Complex()**. **Complex()** creates a **Complex** object for number **0**, and **Complex(a)** creates a **Complex** object with **0** for b . Also provide the **getRealPart()** and **getImaginaryPart()** methods for returning the real part and the imaginary part of the complex number, respectively.

Draw the UML class diagram and implement the class. Use the code at https://liveexample.pearsoncmg.com/test/Exercise13_17.txt to test your implementation. Here is a sample run:

```
Enter the first complex number: 3.5 5.5
Enter the second complex number: -3.5 1
(3.5 + 5.5i) + (-3.5 + 1.0i) = 0.0 + 6.5i
(3.5 + 5.5i) - (-3.5 + 1.0i) = 7.0 + 4.5i
(3.5 + 5.5i) * (-3.5 + 1.0i) = -17.75 + -15.75i
(3.5 + 5.5i) / (-3.5 + 1.0i) = -0.5094 + -1.7i
|(3.5 + 5.5i)| = 6.519202405202649
false
3.5
5.5
[-3.5 + 1.0i, 4.0 + -0.5i, 3.5 + 5.5i, 3.5 + 5.5i]
```

Programming Exercises 561

*** Source Code / Solution :**

```
package Lab8;

public class Complex implements Cloneable, Comparable<Complex>{
    private double a;
```

```
private double b;

// constructors
public Complex(double a, double b) {
    this.a = a;
    this.b = b;
}

public Complex(double a) {
    this(a, 0);
}

public Complex() {
    this(0, 0);
}

// Getter methods
public double getRealPart() {
    return a;
}

public double getImaginaryPart() {
    return b;
}

// Arithmetic operations
public Complex add(Complex other) {
    return new Complex(this.a + other.a, this.b + other.b);
}

public Complex subtract(Complex other) {
    return new Complex(this.a - other.a, this.b - other.b);
}

public Complex multiply(Complex other) {
    double real = this.a * other.a - this.b * other.b;
    double imag = this.b * other.a + this.a * other.b;
    return new Complex(real, imag);
}

public Complex divide(Complex other) {
    double denom = other.a * other.a + other.b * other.b;
    if (denom == 0)
        throw new ArithmeticException("Division by zero");
    double real = (this.a * other.a + this.b * other.b) / denom;
```



```
        double imag = (this.b * other.a - this.a * other.b) / denom;
        return new Complex(real, imag);
    }

    public double abs() {
        return Math.sqrt(a * a + b * b);
    }

    @Override
    public String toString() {
        if (b == 0) {
            return Double.toString(a);
        }

        return "(" + a + " + " + b + "i";
    }

    // implement Cloneable
    @Override
    public Complex clone() {
        try {
            return (Complex) super.clone();
        } catch (CloneNotSupportedException e) {
            return new Complex(this.a, this.b);
        }
    }

    // implement Comparable
    @Override
    public int compareTo(Complex other) {
        return Double.compare(this.abs(), other.abs());
    }
}
```

*** Output:**

```
PS C:\Users\amikam\OneDrive\Learning\Learning\Java Programmin
g\Lab\Lab8> java .\Exercise13_17.java
Enter the first complex number: 1 2
Enter the second complex number: 2 3
((1.0 + 2.0i)) + ((2.0 + 3.0i)) = (3.0 + 5.0i)
((1.0 + 2.0i)) - ((2.0 + 3.0i)) = (-1.0 + -1.0i)
((1.0 + 2.0i)) * ((2.0 + 3.0i)) = (-4.0 + 7.0i)
((1.0 + 2.0i)) / ((2.0 + 3.0i)) = (0.6153846153846154 + 0.076
92307692307693i)
|(1.0 + 2.0i)| = 2.23606797749979
0
1.0
2.0
[(1.0 + 2.0i), (1.0 + 2.0i), (2.0 + 3.0i), (4.0 + -0.5i)]
```

*** Debugging/Testing:**

Bug1: Conflict occurs because Complex class occurs two times.

Fix: Delete the commission part in the Exercise13_17.java to ensure unique Complex.java in the package.

Bug2: The code in the Exercise13_17.java to judge whether two complex is equal has mistaken.

```
Complex c3 = (Complex) c1.clone();
System.out.println(c1 == c3);
```

It compares the reference rather than the two complexes.

Fix:

```
Complex c3 = (Complex) c1.clone();
System.out.println(c1.compareTo(c3));
```